

James Logan · Rust NYC

InterpN: Fast Interpolation



2026-Feb-26

Contents

Rust before Code

Want your program to be faster? Just ask the compiler.

Vectorization

We've been holding it sideways.

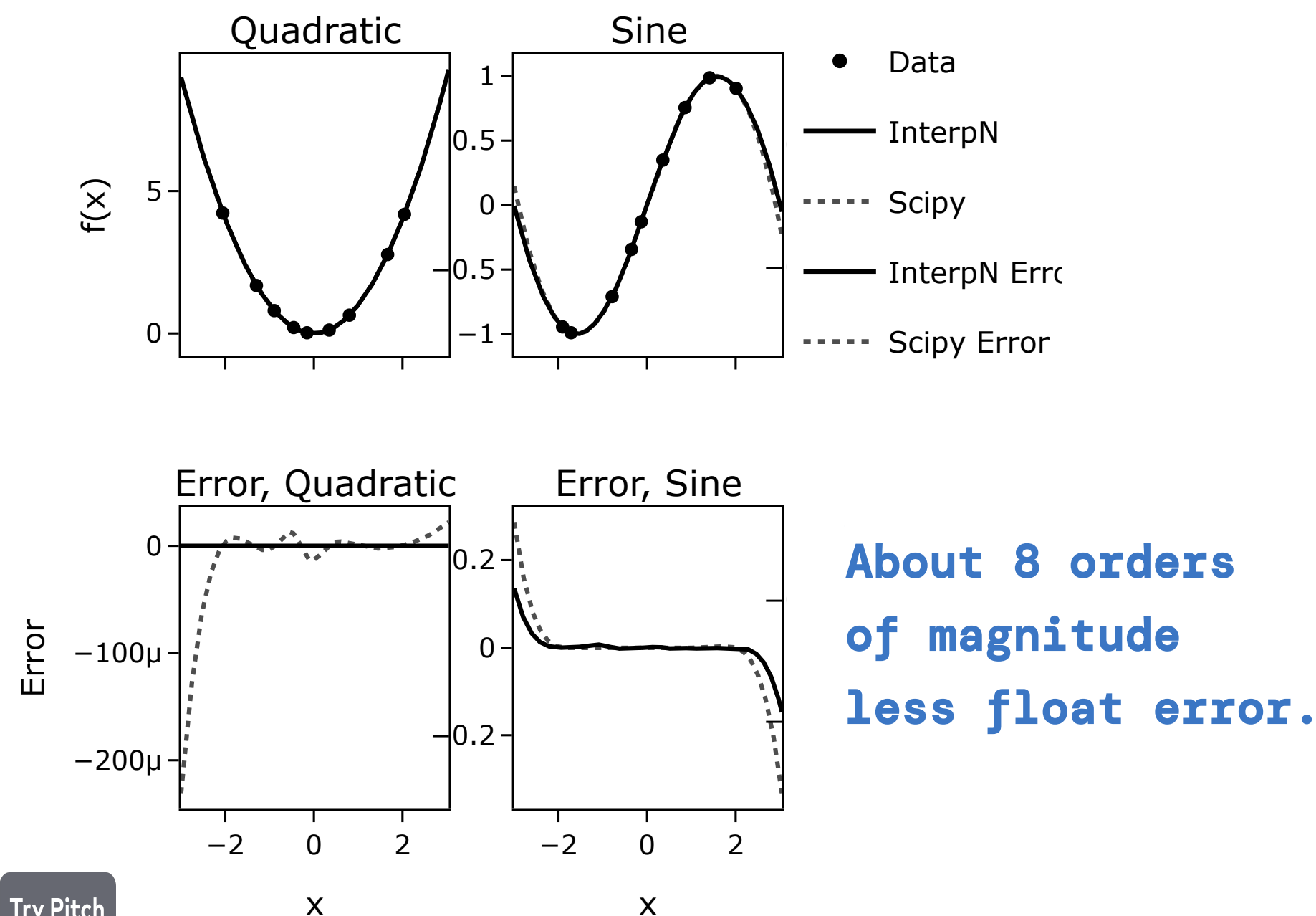
Rust after Code

The myth of premature optimization.

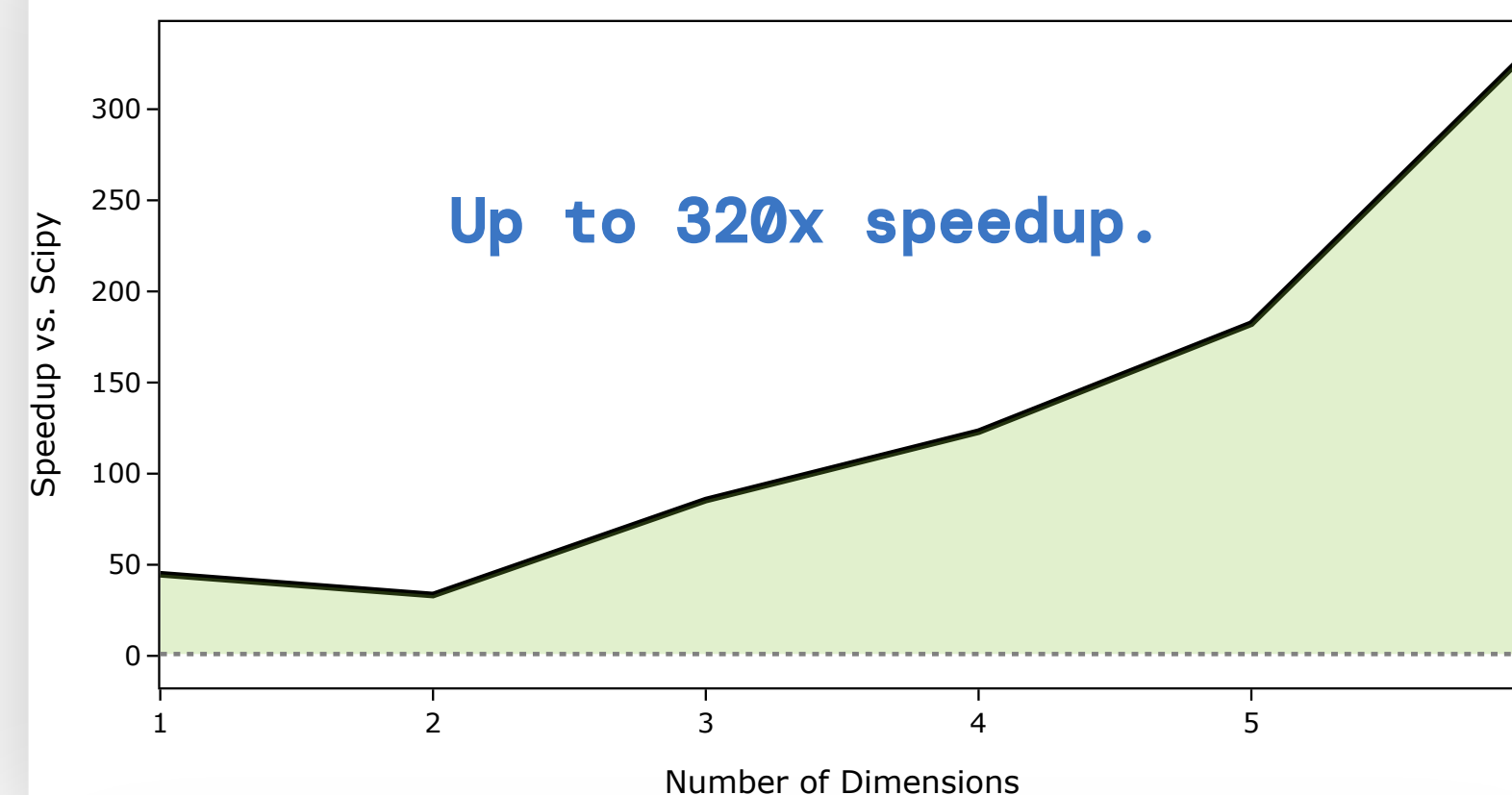
Motivation & Results 1/

We need to estimate the values between data points

- Sometimes data points are expensive
- Sometimes data points have analytic significance
- InterpN supports both embedded *and* scientific computing



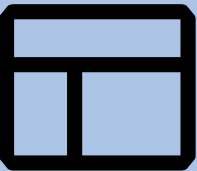
InterpN Speedup vs. Scipy
Linear, 1 Observation Point



Motivation & Results 2/

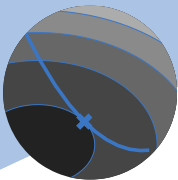
InterpN
(CPU, f64/f32)

1 Sample

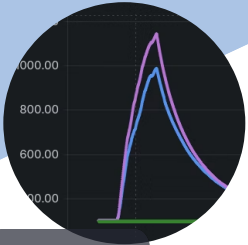


Interactive UI

Optimization,
Contouring,
Isosurfacing

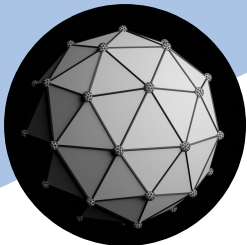


Realtime Data
Acquisition & Controls



**Analog
Circuits**

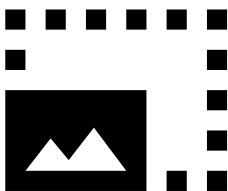
Analysis



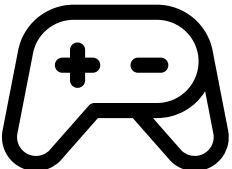
Quantitative Image
Processing



Interactive Image
Processing



10^10 Samples



Graphics



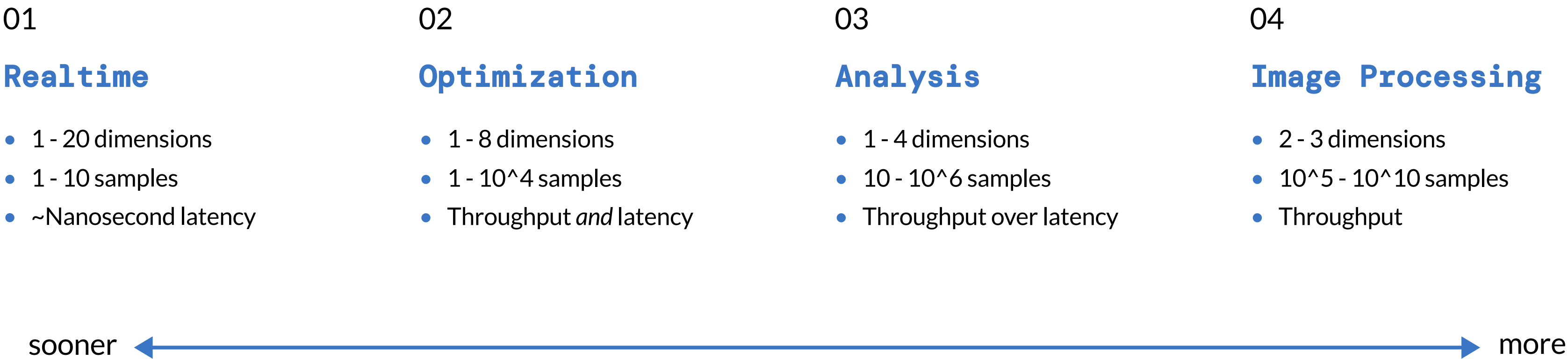
Robotics

Image & Tensor Methods
(GPU/TPU, f16/etc)

Low Latency

Operating Regimes for Interpolators

Motivation & Results 3/

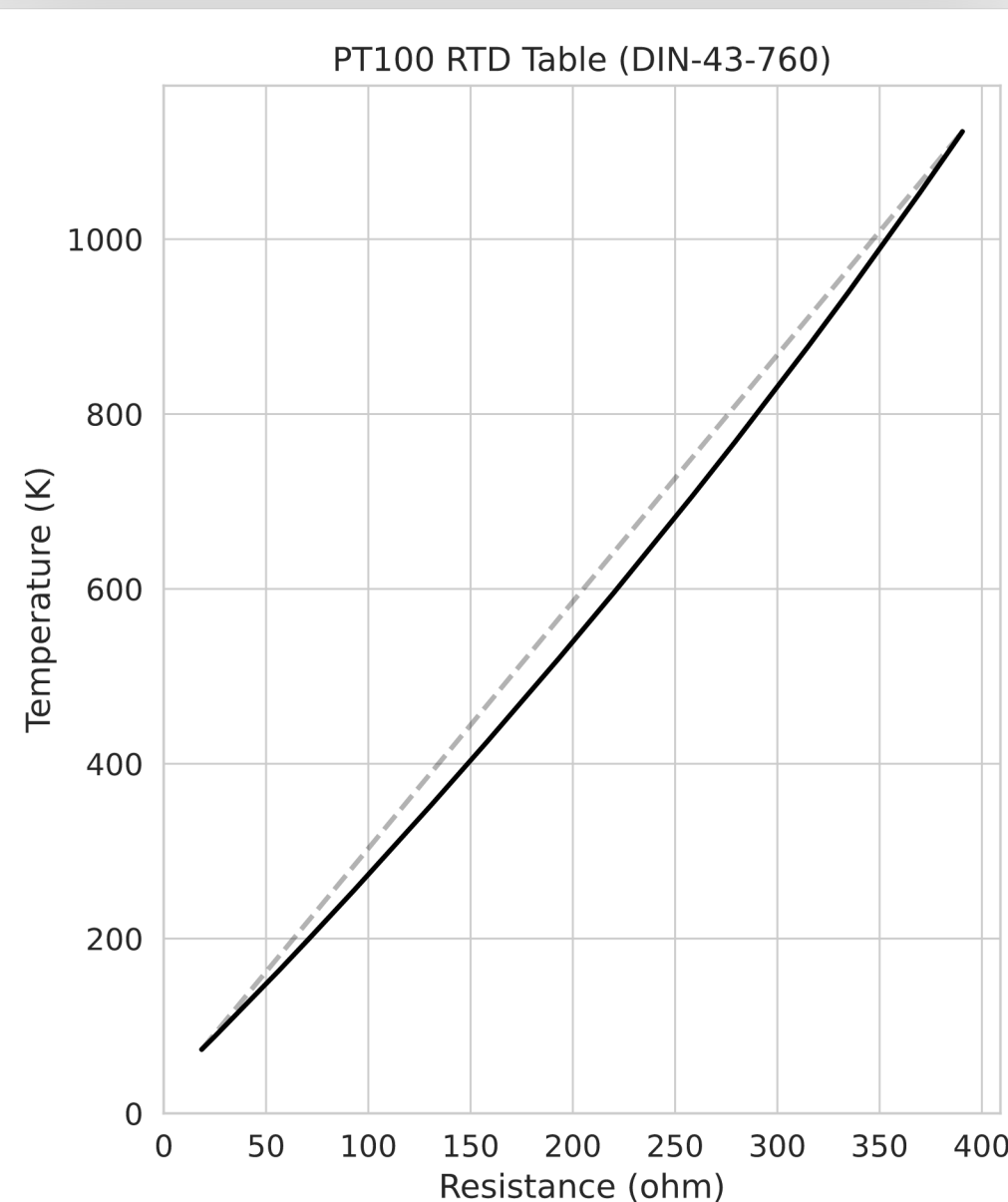
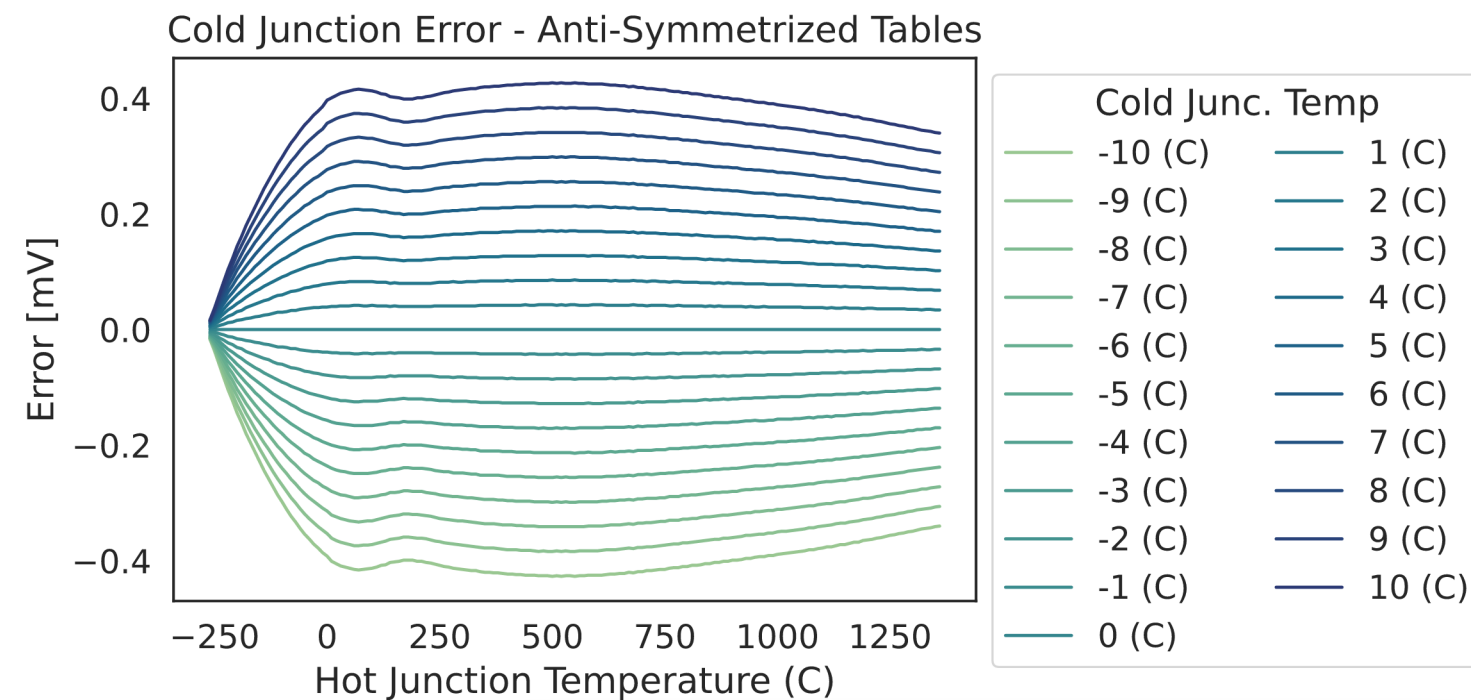


Realtime use-case

Motivation & Results 4/

Realtime applications of interpolation include:

- Thermocouple cold-junction tables (2D).
- RTD & thermistor tables (1D).
- Transducer calibrations (1D - 3D).
- Controller gain tables (1D - 6D).
 - Used to compensate nonlinear plant systems.
 - Higher dimensionalities up to >20 dimensions require neural network interpolators or other dimensionality-reduction techniques.
- Interpolating planned sequences of values on to actual cycle times (1D).
 - Heat treat temperature schedules, for example.
- Adaptive filter coefficient initialization.
- Realtime simulation (“digital twin”) stored inverse sol’ns.



“It Depends”

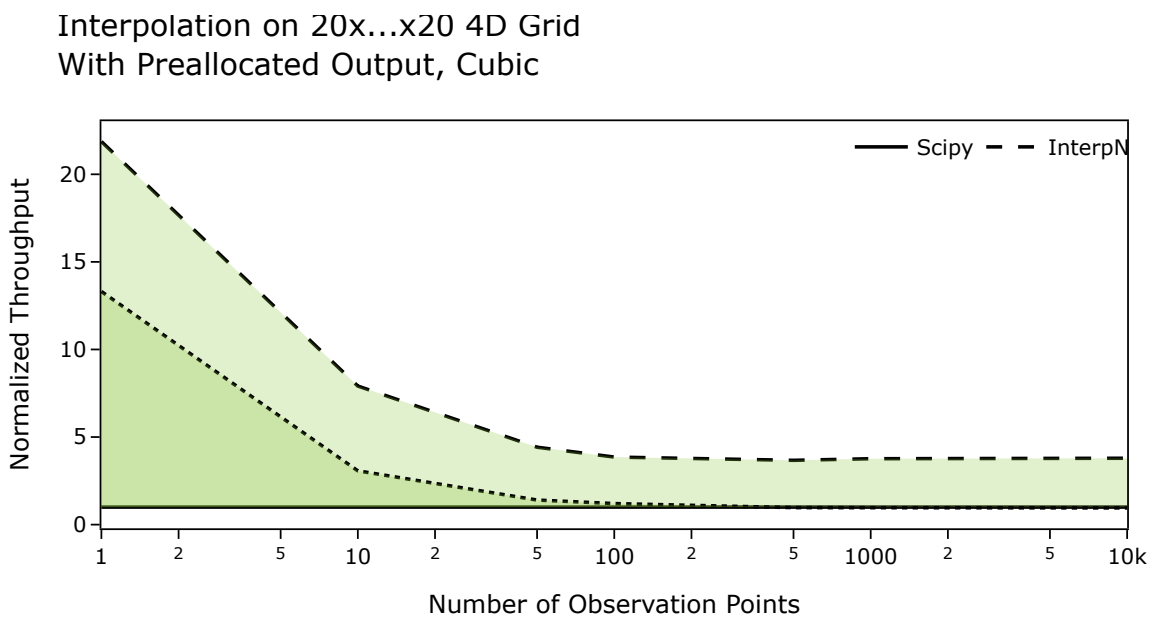
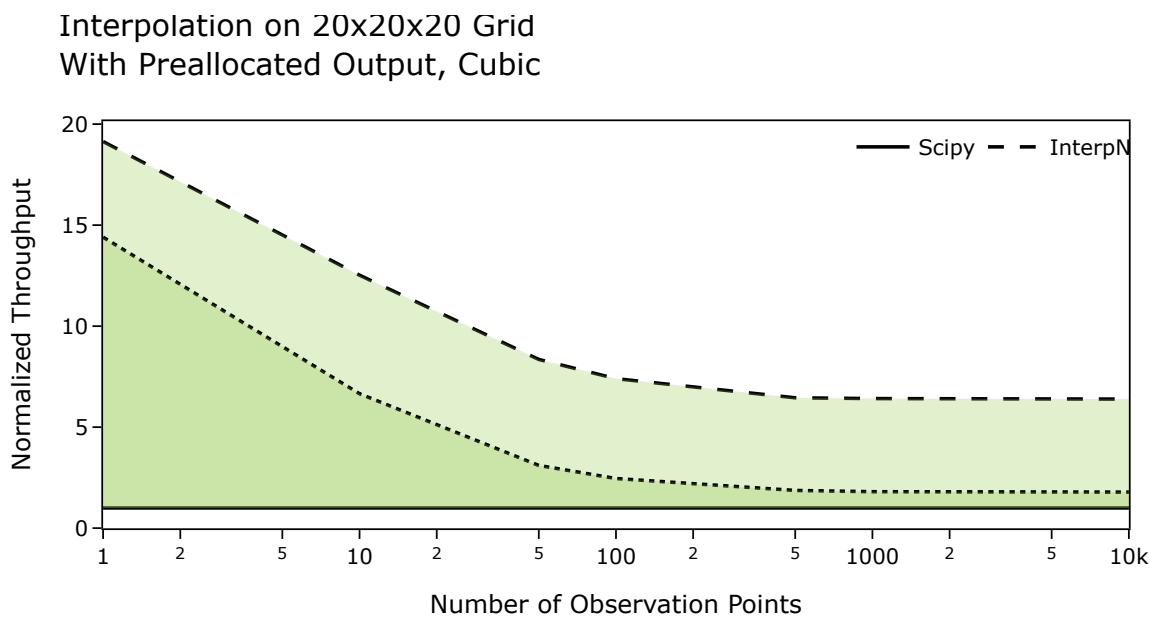
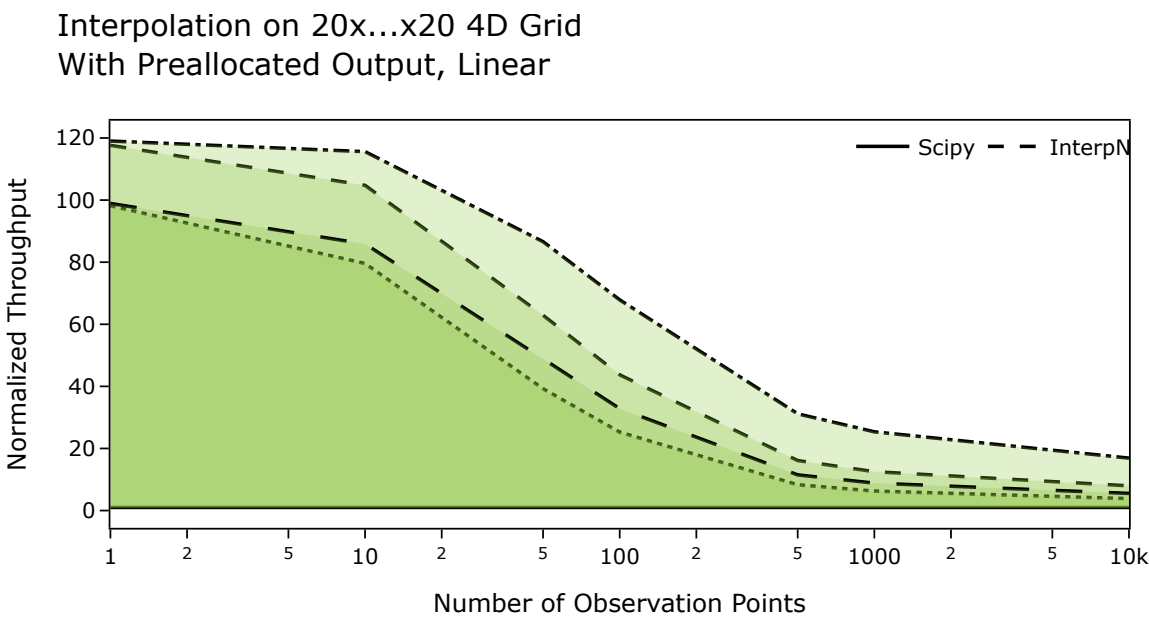
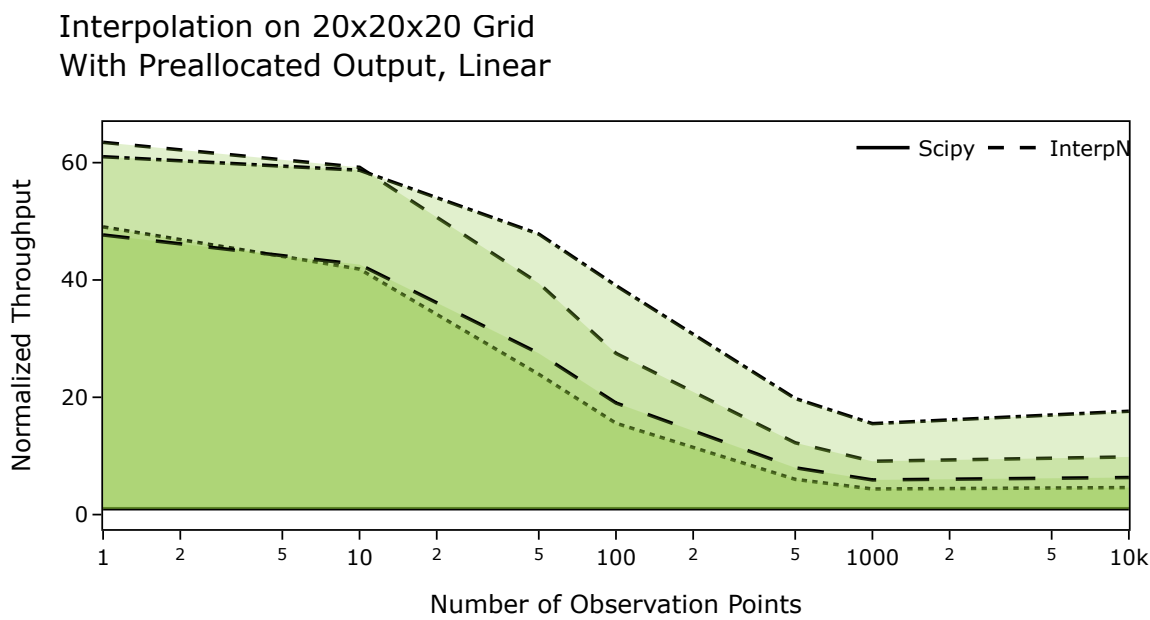
Motivation & Results 5/

more dimensions →

Performance depends on everything!

- There are different
- Functions & regimes
 - Computers
 - Phases of the moon and sun

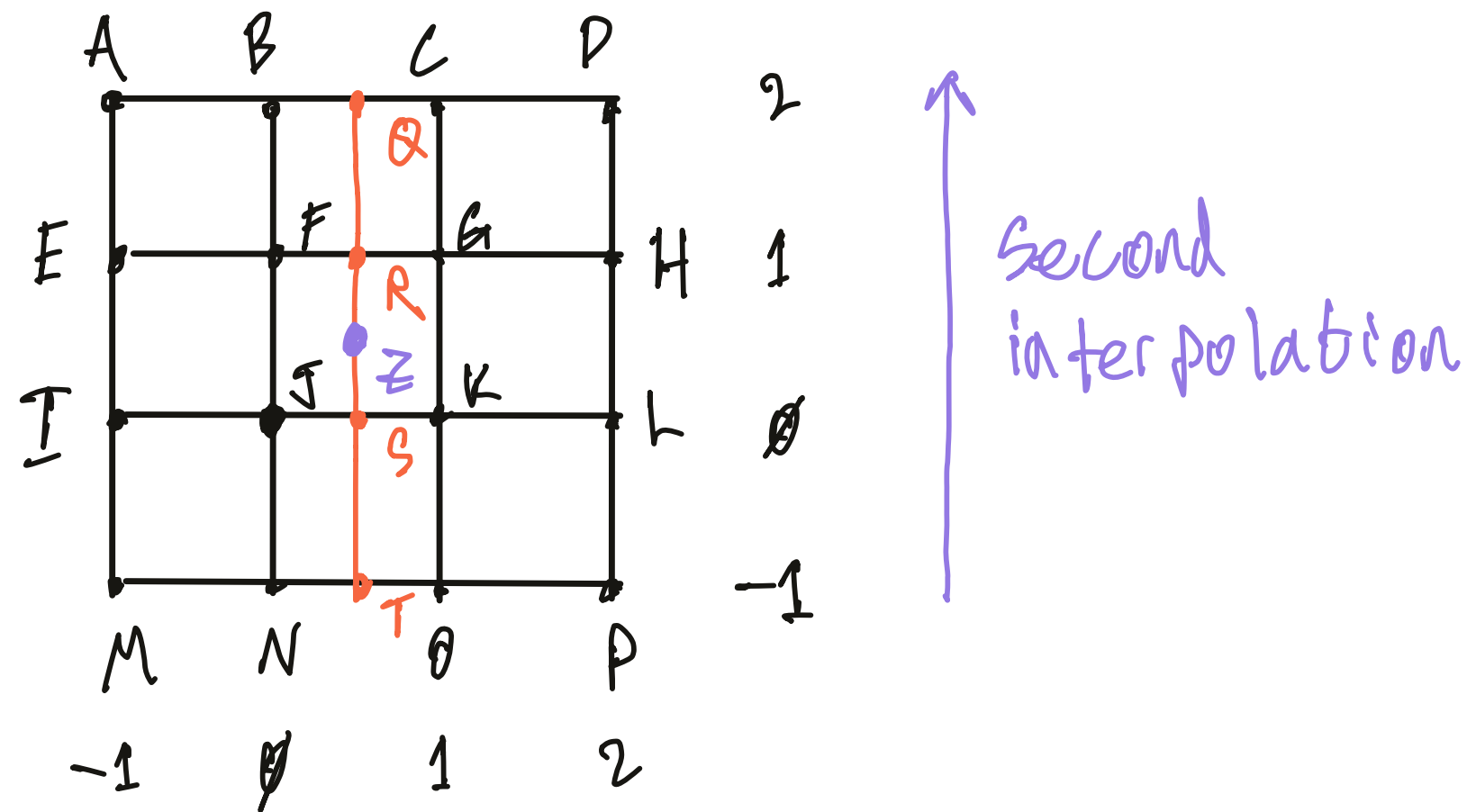
With that said, I think this is okay.



larger kernel ↓

Rust before Code

Want your program to be fast? Just ask the compiler.



Cube → Tree → Loop

The Algorithm

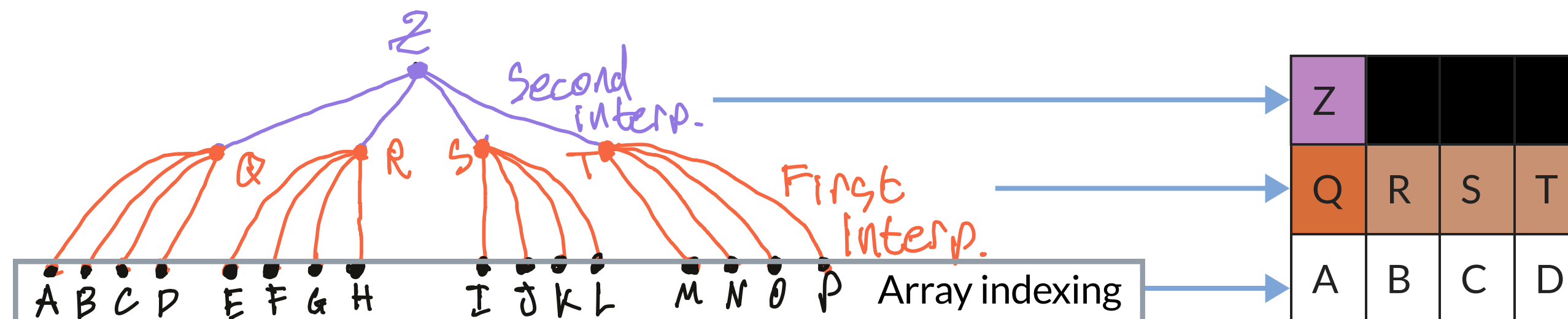
An N-cube around the sample contributes to its value

- X^N can be a lot of data
- Don't want to allocate on a microcontroller

Solution: fixed-size storage on the stack

- No allocation; linear storage instead of exponential

Don't allocate until you have exhausted all other options!



Project Configuration

Cargo.toml

Release Profile

```
[profile.release]
opt-level = 3
codegen-units = 1
lto = true
overflow-checks = true
```

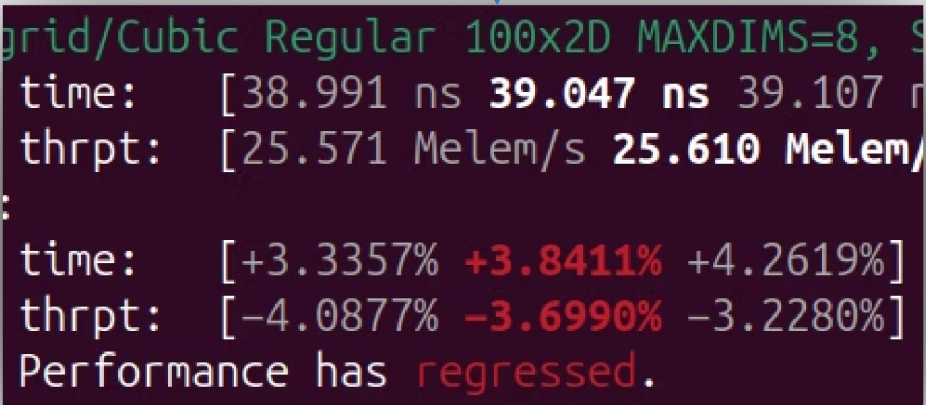
More important for larger projects.

Even for embedded!

Optimize everything at once

Clean up the binary

Free(ish) reliability!



grid/Cubic Regular 100x2D MAXDIMS=8, Sh

time:	[38.991 ns	39.047 ns	39.107 ns]
thrpt:	[25.571 Melem/s	25.610 Melem/s	25.641 Melem/s]

time: [+3.3357% **+3.8411%** +4.2619%]

thrpt: [-4.0877% **-3.6990%** -3.2280%]

Performance has **regressed**.

Try Pitch

.cargo/config.toml

x86 Instruction Sets

```
[target.'cfg(any(target_arch = "x86_64", target_arch = "x64"))']
rustflags = ["-C", "target-cpu=x86-64-v3"]
```

When building for 64-bit x86 targets, use a reference CPU.

See: "Microarchitecture Levels" on wikipedia

#

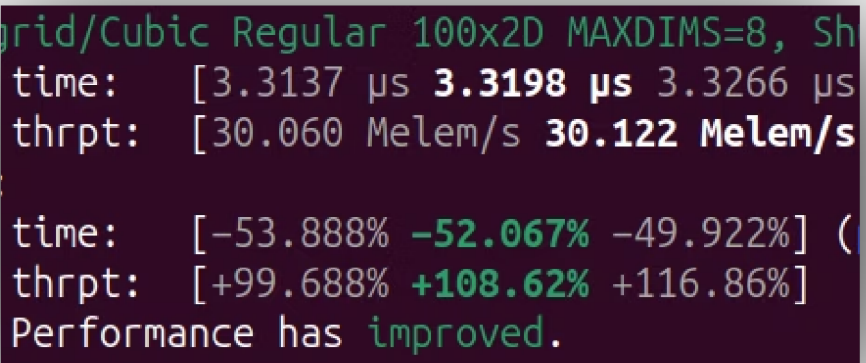
#

x86-64-v1 → 2003

x86-64-v2 → 2008 (SSE4)

x86-64-v3 → 2013 (AVX2, FMA)

x86-64-v4 → 2017 (danger! high latency AVX512)



grid/Cubic Regular 100x2D MAXDIMS=8, Sh

time:	[3.3137 μs	3.3198 μs	3.3266 μs]
thrpt:	[30.060 Melem/s	30.122 Melem/s	30.184 Melem/s]

time: [-53.888% **-52.067%** -49.922%]

thrpt: [+99.688% **+108.62%** +116.86%]

Performance has **improved**.

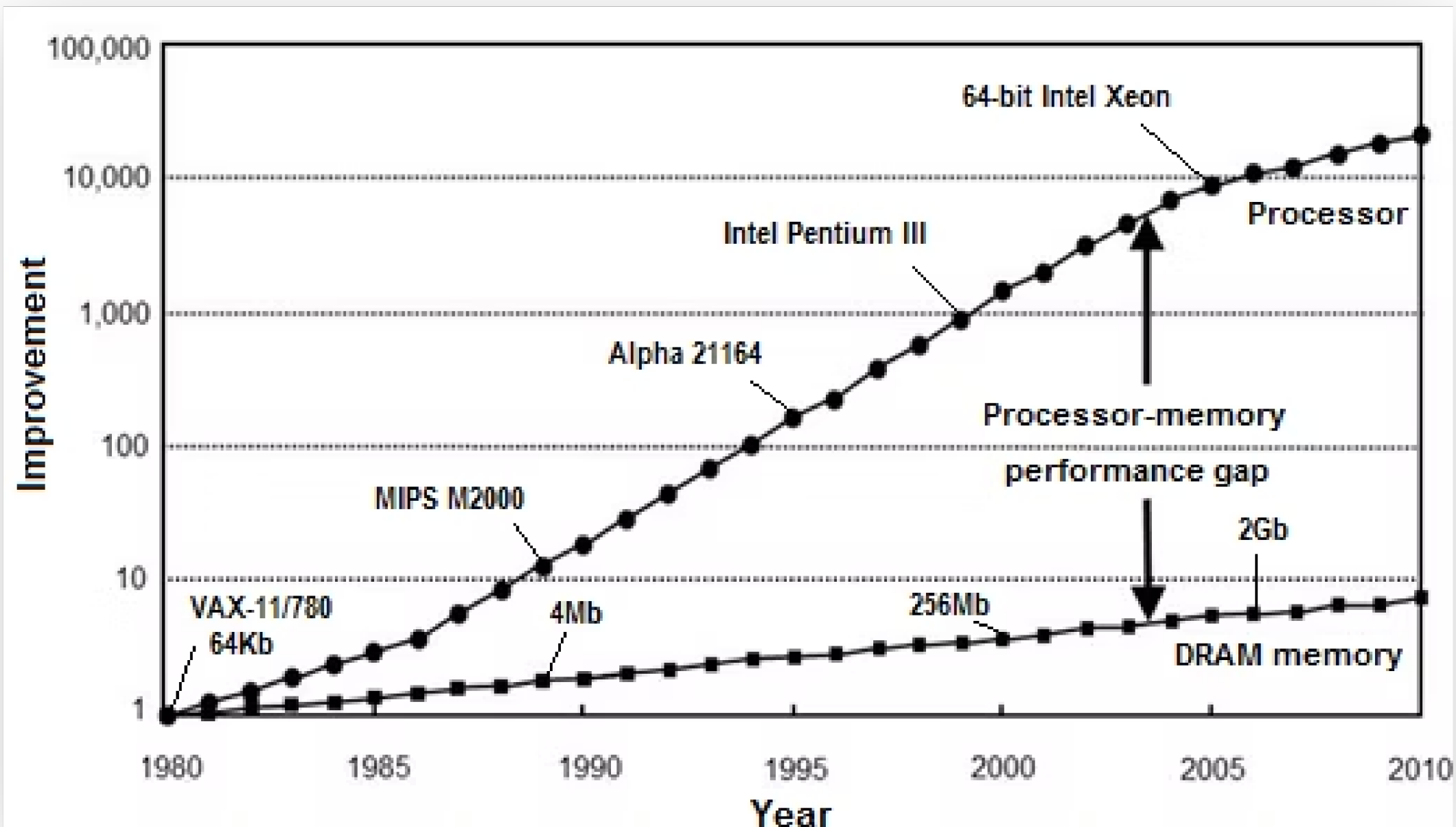
Vectorization

We've been holding it sideways.

Try Pitch

The CPU-RAM gap is not an abstract idea

Vectorization 1/



Things have changed!

RAM is extremely bottlenecked now.

Computing norms were established in the 1990s!

Optimize to reduce RAM traffic over CPU cycles.

D. Efnusheva, A. Cholakoska, and A. Tentov, "A Survey of Different Approaches for Overcoming the Processor - Memory Bottleneck," International Journal of Information Technology and Computer Science, vol. 9, p. 151, Apr. 2017, doi: [10.5121/ijcsit.2017.9214](https://doi.org/10.5121/ijcsit.2017.9214).

Parallelism without threads. It's free compute

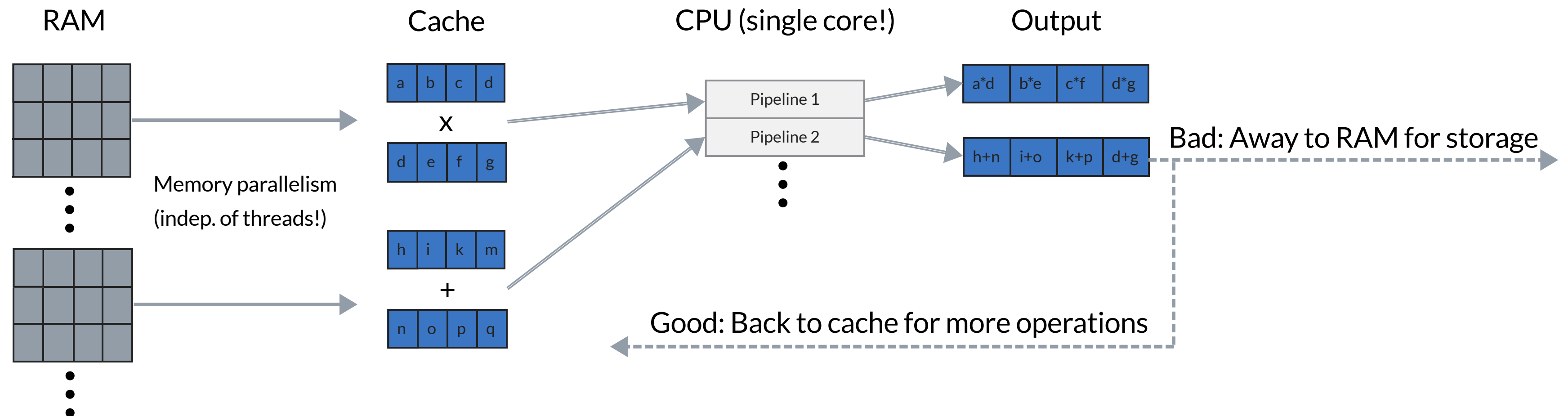
Vectorization 2/

Vectorization

- SIMD / Single Instruction Multiple Data
- Disrupted by branches (including panic branches!)
- Show your code to the compiler!

Pipelining

- CPU can do multiple independent things at once
- Distinct from threading
- Show your code to the CPU!



Vectorization 3/

Type 1: Simple Loops

- ⚠ This produces slow traffic back-and-forth to RAM.
- ⚠ Extra allocations increase latency and total RAM usage.
- ⚠ The compiler and CPU can only optimize one operation at a time.
- ⚠ Bounds checks are repeated for each operation.
- ✅ Formula does not have to be compiled together.

```
fn mul(a: &[f64], b: &[f64]) -> Result<Vec<f64>, &'static str> {  
    // Check bounds before loop!  
    // Otherwise, it will not vectorize  
    let n: usize = a.len();  
    if b.len() != n {  
        return Err("Dimension mismatch");  
    }  
  
    // Allocate storage  
    let mut out: Vec<f64> = vec![0.0; n];  
  
    // Do the calculations  
    for i: usize in 0..n {  
        out[i] = a[i] * b[i];  
    }  
  
    Ok(out)  
}
```

Vectorization 4/

Type 2: Complex Loops

- ✓ Minimum possible RAM accesses and heap allocations.
- ✓ Compiler automatically reuses duplicate values.
- ✓ CPU can execute parallel operations without threading.
- ✓ Bounds checks are only performed once.
- ✓ Compiler generates vector instructions.
- ✓ Branches are okay!
- ✓ Linear speedup when wrapped with threading.
- ⚠ Formula must be compiled together.

Inline everything

```
#[inline]
fn sub(a: [f64; 3], b: [f64; 3]) -> [f64; 3] {
    [a[0] - b[0], a[1] - b[1], a[2] - b[2]]
}
```

```
#[inline]
fn dot(u: [f64; 3], v: [f64; 3]) -> f64 {
    u[0] * v[0] + u[1] * v[1] + u[2] * v[2]
}
```

Try Pitch

```
#[inline]
fn cross(u: [f64; 3], v: [f64; 3]) -> [f64; 3] {
    [
        u[1] * v[2] - u[2] * v[1],
        u[2] * v[0] - u[0] * v[2],
        u[0] * v[1] - u[1] * v[0],
    ]
}
```

```
#[inline]
fn norm(u: [f64; 3]) -> f64 {
    dot(u, u).sqrt()
}
```

scalar formula

```
/// tetrahedron with the first vertex as the origin.
/// https://en.wikipedia.org/wiki/Solid_angle#Tetrahedron
#[inline]
pub fn solid_angle_tetrahedron_scalar(
    v0: [f64; 3],
    v1: [f64; 3],
    v2: [f64; 3],
    v3: [f64; 3],
) -> f64 {
    // Vertex vectors
    let (a, b, c) = (sub(v1, v0), sub(v2, v0), sub(v3, v0)); // (m)
    let (la, lb, lc) = (norm(a), norm(b), norm(c)); // (m) Vertex vector lengths
    let abc = la * lb * lc; // (m^3) Length product. Branch determined here!

    // Solid angle
    let triple = dot(a, cross(b, c)); // (m^3) Scalar triple product
    let denom = abc + dot(a, b) * lc + dot(a, c) * lb + dot(b, c) * la; // (m^3)
    let angle = 2.0 * libm::atan2(triple, denom); // (rad) f64::atan2 defers to libc

    // Check for degeneracy _last_ to avoid disrupting flow
    if abc != 0.0 {
        angle
    } else {
        0.0
    }
}
```

vector formula

```
/// Vector variant of [solid_angle_tetrahedron_scalar]
#[inline] // Enable cross-crate inlining
pub fn solid_angle_tetrahedron(
    tetrahedra: &[[f64; 3]; 4],
    out: &mut [f64],
) -> Result<(), &'static str> {
    // Check bounds
    let n = out.len();
    if tetrahedra.len() != n {
        return Err("Dimension mismatch");
    }

    // Do calculations
    for i in 0..n {
        let tet = tetrahedra[i];
        out[i] = solid_angle_tetrahedron_scalar(tet[0], tet[1], tet[2], tet[3]);
    }

    Ok(())
}
```

How do we know if it worked?

Vectorization 5/

Benchmarks

Mess it up and see if it gets worse!

Fix it and see if it gets better!

`criterion` is great for this.

bounds checks!!

```
ed_100-grid/Linear Regular 100x1D MAXDIMS=1, Shuffled Order/1
time: [7.5980 ns 7.6594 ns 7.7262 ns]
thrpt: [129.43 Melem/s 130.56 Melem/s 131.61 Melem/s]
change:
time: [+35.729% +37.337% +38.877%] (p = 0.00 < 0.05)
thrpt: [-27.994% -27.186% -26.324%]
Performance has regressed.
```

```
ed_100-grid/Linear Regular 100x1D MAXDIMS=1, Shuffled Order/1
time: [5.6419 ns 5.6935 ns 5.7460 ns]
thrpt: [174.03 Melem/s 175.64 Melem/s 177.24 Melem/s]
change:
time: [-26.692% -25.793% -24.916%] (p = 0.00 < 0.05)
thrpt: [+33.184% +34.757% +36.411%]
Performance has improved.
```

Assembly

Go inside and look!

- Single file: use Godbolt!
- Multi-file: use cargo-show-asm!
- Look for `panic` and packed ops.

optimizations!!

```
pub fn solid_angle_tetrahedron(
    tetrahedra: &[[f64; 3]; 4],
    out: &mut [f64],
) -> Result<(), &'static str> {
    // Check bounds
    let n = out.len();
    if tetrahedra.len() != n {
        return Err("Dimension mismatch");
    }

    // Do calculations
    for i in 0..n {
        let tet = tetrahedra[i];
        out[i] = solid_angle_tetrahedron_scalar(tet[0],
    }

    ok(())
}
```

```
17
18 .LBE
19 lea rbp, [r13
20 vmovsd xmm0, qwo
21 vmovupd xmm4, xmm
22 vmovupd xmm7, xmm
23 vmovupd xmm3, xmm
24 vmovupd xmm6, xmm
25 vmovupd xmm8, xmm
26 vmovupd xmm9, xmm
27 vsubsd xmm1, xmm
28 vmovsd xmm0, qwo
29 vmovhpd xmm0, xmm
30 vshufpd xmm2, xmm
31 vsubpd xmm2, xmm
32 vsubpd xmm5, xmm
33 vsubpd xmm3, xmm
34 vsubpd xmm6, xmm
```

```

/// tetrahedron with the first vertex as the origin.
/// https://en.wikipedia.org/wiki/Solid_angle#Tetrahedron
#[inline]
pub fn solid_angle_tetrahedron_scalar(
    v0: [f64; 3],
    v1: [f64; 3],
    v2: [f64; 3],
    v3: [f64; 3],
) -> f64 {
    // Vertex vectors
    let (a, b, c) = (sub(v1, v0), sub(v2, v0), sub(v3, v0)); // (m)
    let (la, lb, lc) = (norm(a), norm(b), norm(c)); // (m) Vertex vector lengths
    let abc = la * lb * lc; // (m^3) Length product. Branch determined here!

    // Solid angle
    let triple = dot(a, cross(b, c)); // (m^3) Scalar triple product
    let denom = abc + dot(a, b) * lc + dot(a, c) * lb + dot(b, c) * la; // (m^3)
    let angle = 2.0 * libm::atan2(triple, denom); // (rad) f64::atan2 defers to libc

    // Check for degeneracy _last_ to avoid disrupting flow
    if abc != 0.0 {
        angle
    } else {
        0.0
    }
}

```

branch!! in my loop??

```

/// Vector variant of [solid_angle_tetrahedron_scalar]
#[inline] // Enable cross-crate inlining
pub fn solid_angle_tetrahedron(
    tetrahedra: &[[f64; 3]; 4],
    out: &mut [f64],
) -> Result<(), &'static str> {
    // Check bounds
    let n = out.len();
    if tetrahedra.len() != n {
        return Err("Dimension mismatch");
    }

    // Do calculations
    for i in 0..n {
        let tet = tetrahedra[i];
        out[i] = solid_angle_tetrahedron_scalar(tet[0], tet[1], tet[2], tet[3]);
    }
}

```

A branch is early or late, never on time

Vectorization 6/

✓ **Early return is ok...**

but needs to be very early to avoid disrupting vectorization.

Must be before both the loop and the scalar calc!

Bounds checks are in this category.

✗ **Middle return is not ok!!**

Can't vectorize if we're not sure the calculations will happen!

Ambiguity means low performance.

Don't surprise the CPU!

✓ **Late return is ok...**

as long as the *condition* for the branch is determined early.

Scalar calculations should be unconditional.

Only the final return should be conditional!

Conditional selection is a CPU instruction on most arches!

A word about allocation

Vectorization 7/

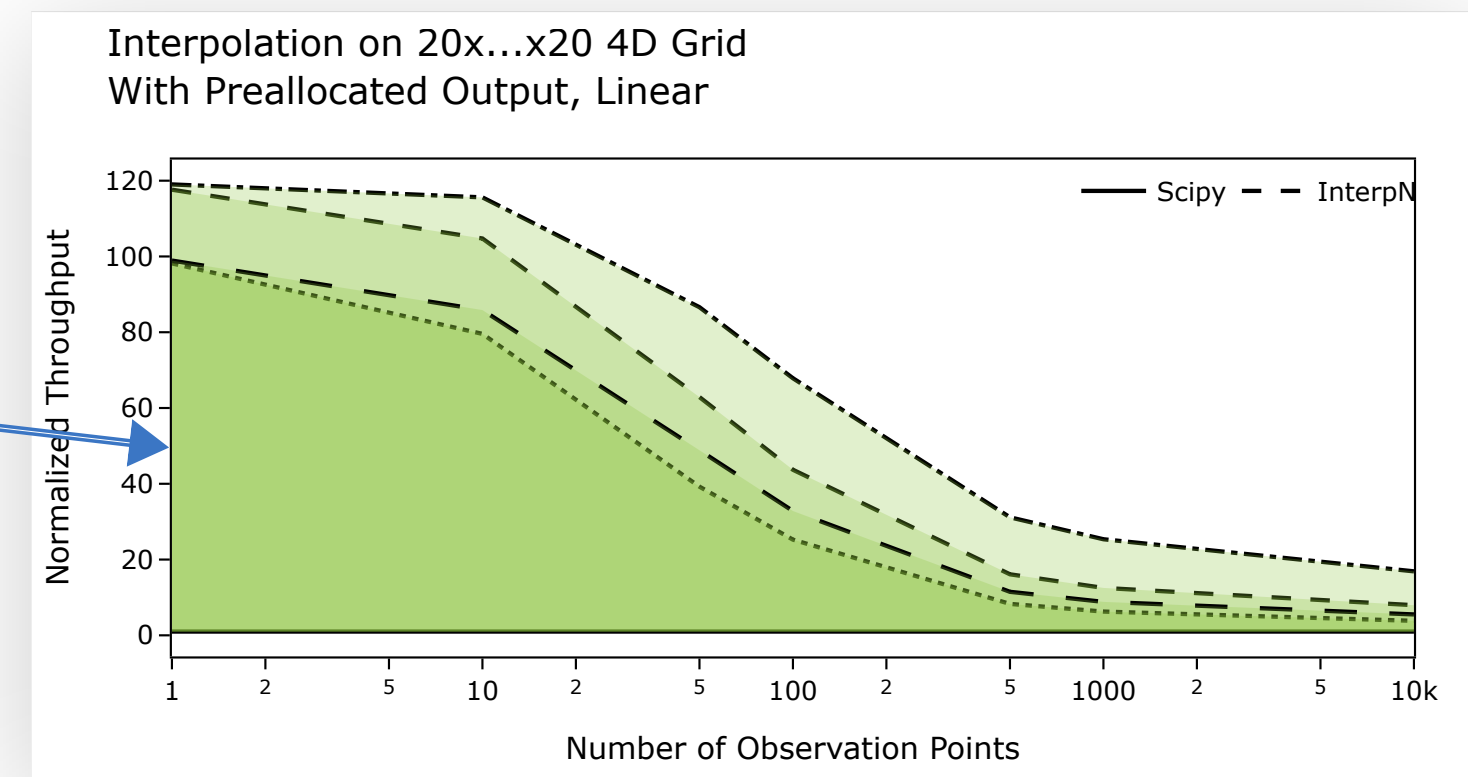
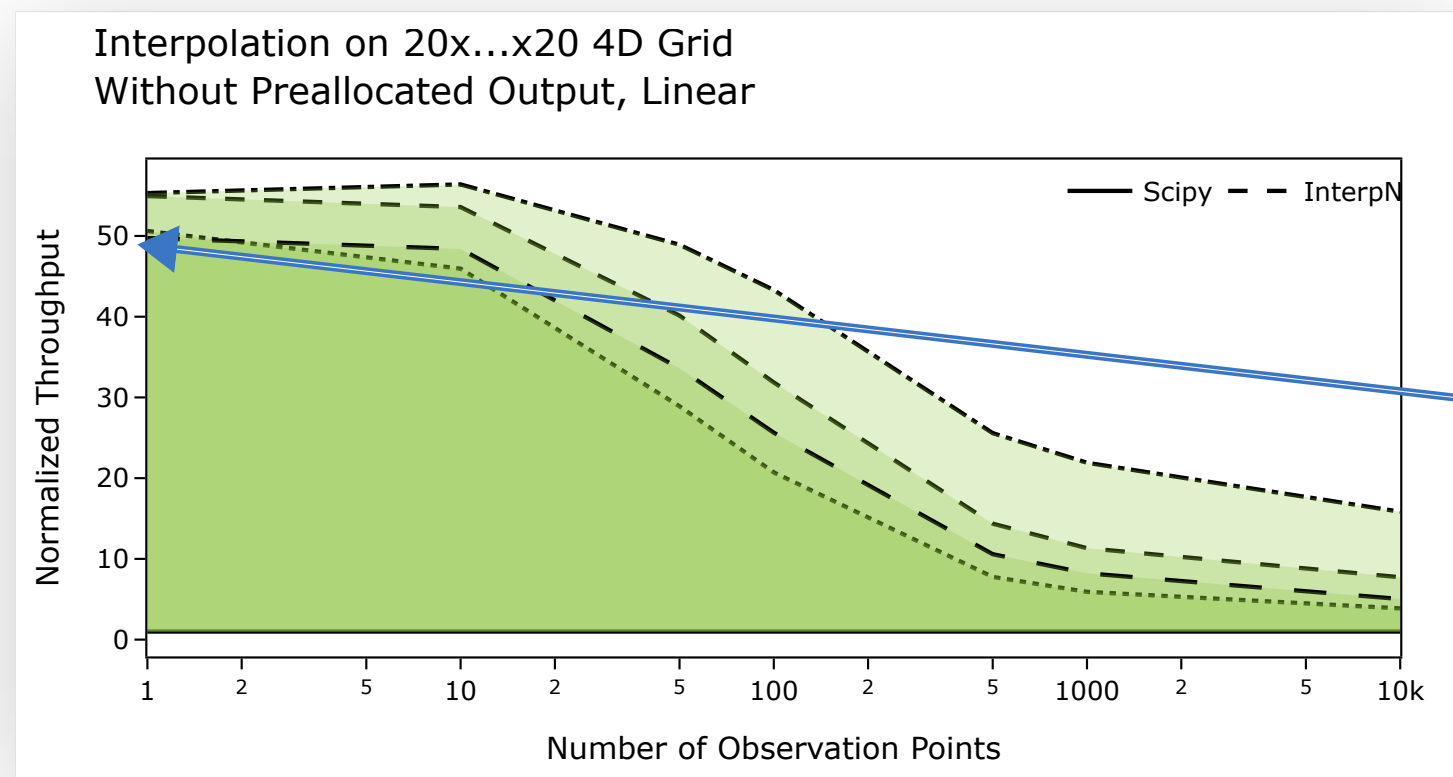
Allocation is the enemy

- Effect depends on exact use pattern; **masked by benchmarking**.
- Inter-thread and inter-process interference & extra RAM accesses.

At least give users the option to pre-allocate output!

2x speedup for small sizes.

Just stop allocating!



Telling rustc what you know about the program

Compile-Time Loop Unrolling 1/

- Problem: house is too cold
- Solution: unroll large loops with `crunchy`

This can be a const generic bound!

```
if N <= 3 {  
    unroll! {  
        for i < 64 in 0..nverts { // const loop  
            unroll_vertices_body!(i);  
        }  
    }  
} else {  
    for i: usize in 0..nverts {  
        unroll_vertices_body!(i);  
    }  
}
```

```
Grid/Cubic Regular 100x2D MAXDIMS=8, Sh  
time: [3.1936 µs 3.1989 µs 3.2054 µs  
thrpt: [31.197 Melem/s 31.260 Melem/s  
  
time: [-31.332% -30.524% -29.658%] (  
thrpt: [+42.163% +43.934% +45.628%]  
Performance has improved.
```

rustc can handle small loops without help

Compile-Time Loop Unrolling 2/

```
// Index, interpolate, or pass on each level of the tree
unroll!{
  for j < 5 in 0..N { // const loop

    // Most of these iterations will get optimized out
    if const{j == 0} { // const branch
      // At leaves, index values

      unroll!{
        for k < 5 in 0..N { // const loop
          // Bit pattern in an integer matches C-ordered array indexing
          // so we can just use the vertex index to index into the array
          // by selecting the appropriate bit from the index.
          const OFFSET: usize = const{(i & (3 << (2*k))) >> (2*k)};
          loc[k] = origin[k] + OFFSET;
        }
        const STORE_IND: usize = i % FP;
        store[0][STORE_IND] = index_arr_fixed_dims(loc, dimprod, self.vals);
      }
    } else { // const branch
      // For other nodes, interpolate on child values

      const Q: usize = const{FP.pow(j as u32)};
      const LEVEL: bool = const {(i + 1).is_multiple_of(Q)};
      const P: usize = const{((i + 1) / Q).saturating_sub(1) % FP};
      const IND: usize = const{j.saturating_sub(1)};

      if LEVEL { // const branch
        let interped = interp_inner::<T>(&store[IND],
          dts[IND],
          sat[IND],
          self.linearize_extrapolation
        );

        store[j][P] = interped;
      }
    }
  }
}
```

More unrolling
Longer compile time

```
// Index, interpolate, or pass on each level of the tree
for j in 0..N {
  // Most of these iterations will get optimized out
  if j == 0 {
    // const branch
    // At leaves, index values
    for k in 0..N {
      // Bit pattern in an integer matches C-ordered array indexing
      // so we can just use the vertex index to index into the array
      // by selecting the appropriate bit from the index.
      let offset: usize = ($i & (3 << (2 * k))) >> (2 * k);
      loc[k] = origin[k] + offset;
    }
    let store_ind: usize = $i % FP;
    store[0][store_ind] = index_arr_fixed_dims(loc, dimprod, self.vals);
  } else {
    // const branch
    // For other nodes, interpolate on child values
    let q: usize = FP.pow(j as u32);
    let level: bool = ($i + 1).is_multiple_of(q);
    let p: usize = (($i + 1) / q).saturating_sub(1) % FP;
    let ind: usize = j.saturating_sub(1);

    if level {
      // const branch
      let interped = interp_inner::<T>(&store[ind],
        dts[ind],
        sat[ind],
        self.linearize_extrapolation,
      );

      store[j][p] = interped;
    }
  }
}
```

Less unrolling
Shorter compile time

Same performance!

grid/Cubic Regular 100x2D MAXDIMS=8, Shu			
time:	[30.928 ms	31.138 ms	31.473 ms]
thrpt:	[31.774 Melem/s	32.115 Melem/s	
:			
time:	[-1.8841%	-1.0393%	-0.0013%]
thrpt:	[+0.0013%	+1.0503%	+1.9203%]
Change within noise threshold.			

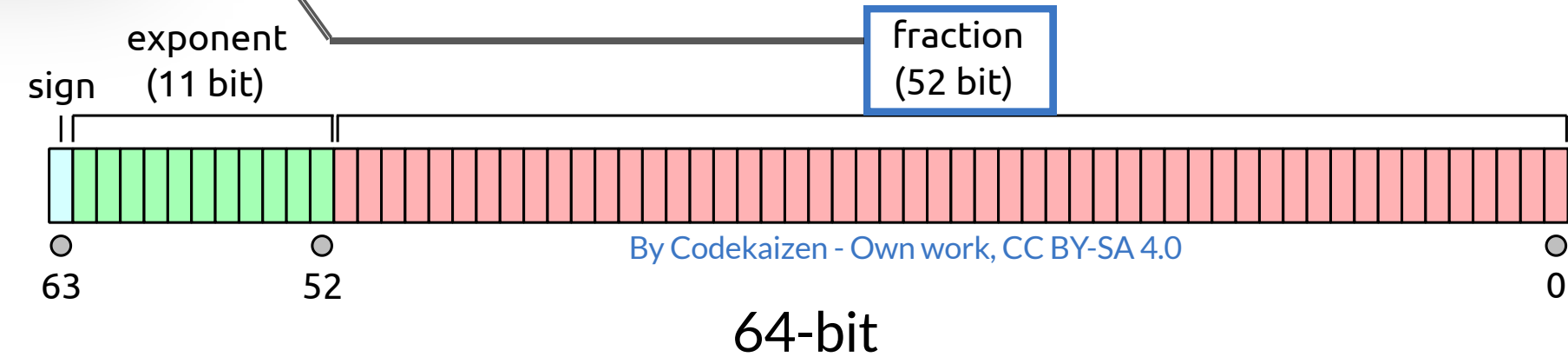
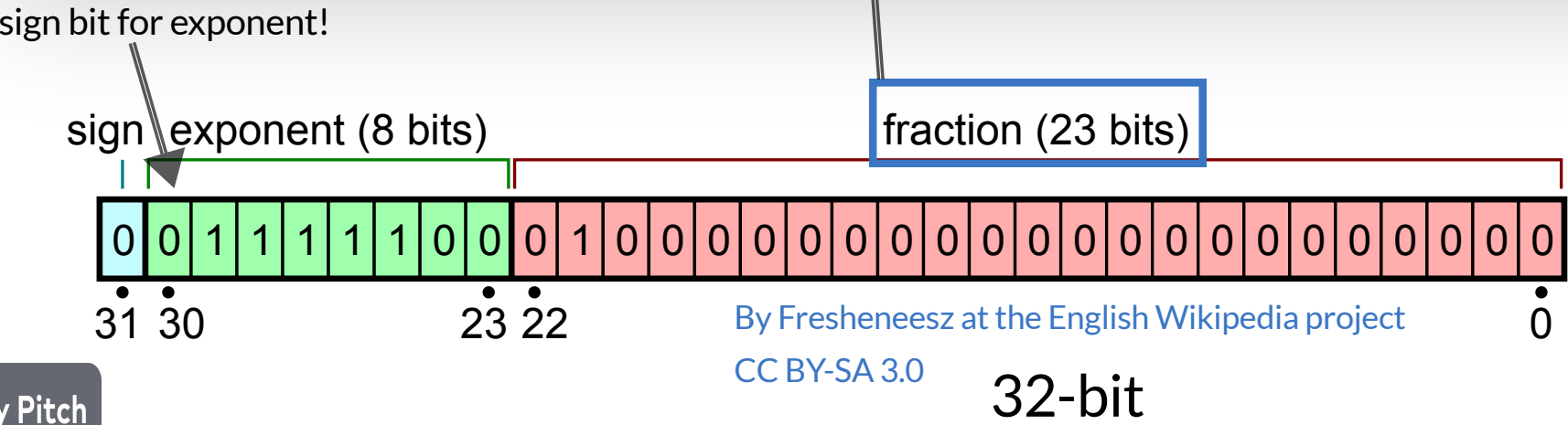
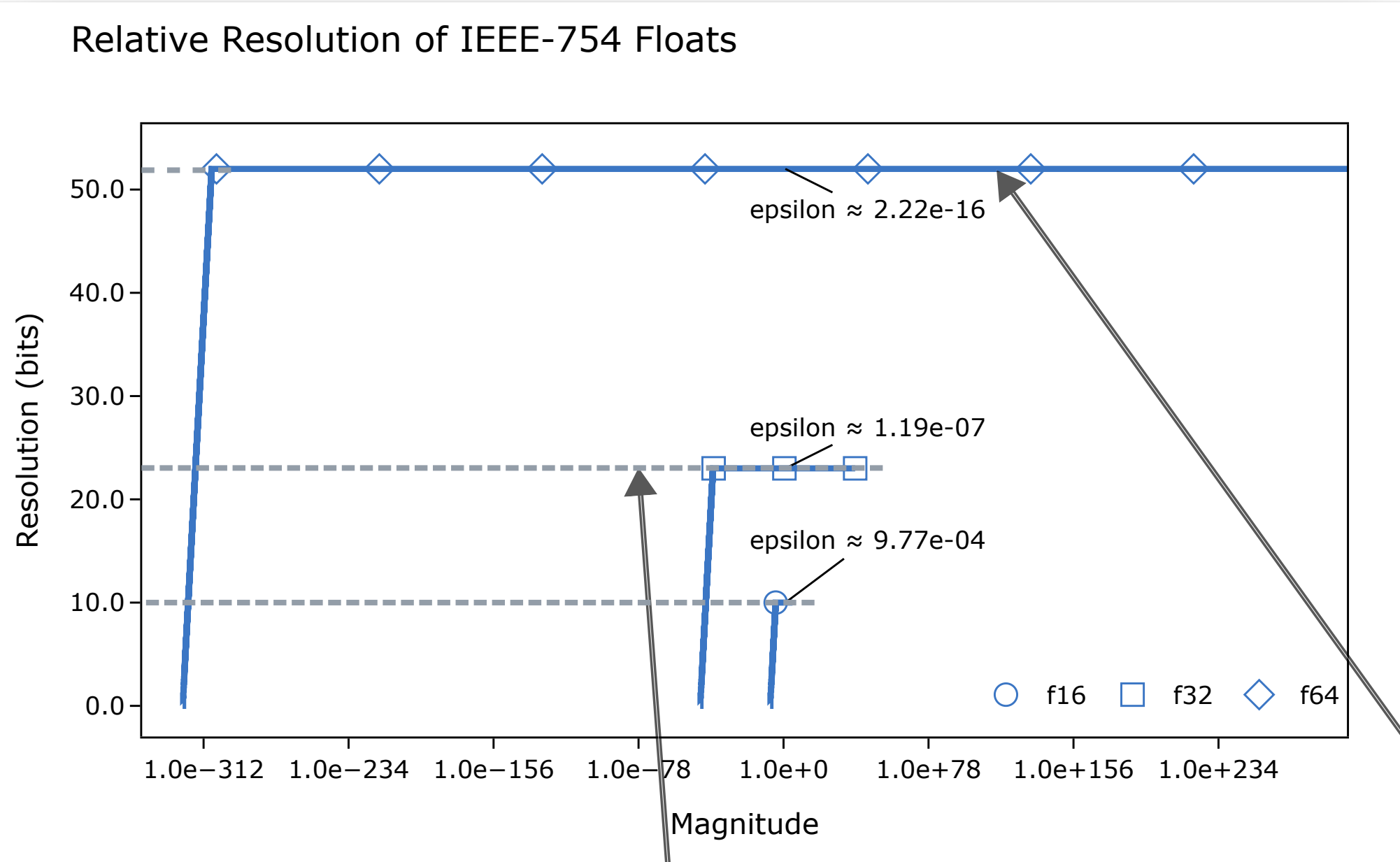
Two flops for the price of one (ish)

Fused Multiply-Add (FMA) 1/

- Good for application code, not for embedded
- ``a.mul_add(x, b)`` → ``a*x + b`` in a single step.
 - Less roundoff error. Float resolution is real!
 - Some subtlety; still worth it.

- Demystify floats
- It's just a (near-)constant relative resolution!
 - You can do error accounting in your head!
 - Do not abide a -ffast-math

$$e = 0 \Rightarrow x = (-1)^s \left(\sum_{i=1}^{52} b_i 2^{-i} \right) 2^{-1022}$$
$$1 \leq e \leq 2046 \Rightarrow x = (-1)^s \left(1 + \sum_{i=1}^{52} b_i 2^{-i} \right) 2^{e-1023}$$
$$e = 2047 \Rightarrow x = \pm \infty \text{ or NaN}$$



The return of the solid angle

Fused Multiply-Add (FMA) 2/

```
#[inline]
fn dot(u: [f64; 3], v: [f64; 3]) -> f64 {
    u[0] * v[0] + u[1] * v[1] + u[2] * v[2]
}

#[inline]
fn cross(u: [f64; 3], v: [f64; 3]) -> [f64; 3] {
    [
        u[1] * v[2] - u[2] * v[1],
        u[2] * v[0] - u[0] * v[2],
        u[0] * v[1] - u[1] * v[0],
    ]
}

#[inline]
fn norm(u: [f64; 3]) -> f64 {
    dot(u, u).sqrt()
}

/// Angular portion of a sphere subtended by a
/// tetrahedron with the first vertex as the origin.
/// https://en.wikipedia.org/wiki/Solid\_angle#Tetrahedron
#[inline]
pub fn solid_angle_tetrahedron_scalar(
    v0: [f64; 3],
    v1: [f64; 3],
    v2: [f64; 3],
    v3: [f64; 3],
) -> f64 {
    // Vertex vectors
    let (a, b, c) = (sub(v1, v0), sub(v2, v0), sub(v3, v0)); // (m)
    let (la, lb, lc) = (norm(a), norm(b), norm(c)); // (m) Vertex vector lengths
    let abc = la * lb * lc; // (m^3) Length product. Branch determined here!

    // Solid angle
    let triple = dot(a, cross(b, c)); // (m^3) Scalar triple product
    let denom = abc + dot(a, b) * lc + dot(a, c) * lb;
    let angle = 2.0 * libm::atan2(triple, denom);

    // Check for degeneracy _last_ to avoid disrupting flow
    if abc != 0.0 {
        angle
    } else {
        0.0
    }
}
```

No FMA

```
#[inline]
fn dot(u: [f64; 3], v: [f64; 3]) -> f64 {
    u[0].mul_add(v[0], u[1].mul_add(v[1], u[2] * v[2]))
}

#[inline]
fn cross(u: [f64; 3], v: [f64; 3]) -> [f64; 3] {
    [
        u[1].mul_add(v[2], -u[2] * v[1]),
        u[2].mul_add(v[0], -u[0] * v[2]),
        u[0].mul_add(v[1], -u[1] * v[0]),
    ]
}

#[inline]
fn norm(u: [f64; 3]) -> f64 {
    dot(u, u).sqrt()
}

/// Angular portion of a sphere subtended by a
/// tetrahedron with the first vertex as the origin.
/// https://en.wikipedia.org/wiki/Solid\_angle#Tetrahedron
#[inline]
pub fn solid_angle_tetrahedron_scalar(
    v0: [f64; 3],
    v1: [f64; 3],
    v2: [f64; 3],
    v3: [f64; 3],
) -> f64 {
    // Vertex vectors
    let (a, b, c) = (sub(v1, v0), sub(v2, v0), sub(v3, v0)); // (m)
    let (la, lb, lc) = (norm(a), norm(b), norm(c)); // (m) Vertex vector lengths
    let abc = la * lb * lc; // (m^3) Length product. Branch determined here!

    // Solid angle
    let triple = dot(a, cross(b, c)); // (m^3) Scalar triple product
    let denom = dot(a, b).mul_add(lc, dot(a, c).mul_add(lb, dot(b, c).mul_add(la, abc))); // (m^3)
    let angle = 2.0 * libm::atan2(triple, denom); // (rad) f64::atan2 defers to libc

    // Check for degeneracy _last_ to avoid disrupting flow
    if abc != 0.0 {
        angle
    } else {
        0.0
    }
}
```

Yes FMA

rustc 1.92.0 -O -C target-cpu=x86-64-v3

1 of 18

18% Vector FMA

```
38 vfm(add|sub)
39 vmulpd xmm0, xmm8, xmm8
40 vfmadd231pd xmm0, xmm6, xmm6
41 vfmadd231pd xmm0, xmm4, xmm4
42 vsqrtpd xmm5, xmm0
43 vmulsd xmm0, xmm1, xmm1
44 vfmadd231sd xmm0, xmm3, xmm3
45 vfmadd231sd xmm0, xmm3, xmm3
46 vsqrtsd xmm7, xmm0
47 vshufpd xmm9, xmm5, xmm5, 1
48 vmulsd xmm0, xmm9, xmm5
49 vmulsd xmm15, xmm0, xmm7
50 vmovsd qword ptr [rsp], xmm15
51 vshufpd xmm10, xmm8, xmm8, 1
52 vmulsd xmm11, xmm10, xmm3
53 vshufpd xmm12, xmm6, xmm6, 1
54 vfmsub231sd xmm11, xmm12, xmm1
55 vshufpd xmm13, xmm4, xmm4, 1
56 vmulsd xmm14, xmm13, xmm1
57 vfmsub231sd xmm14, xmm10, xmm2
58 vmulsd xmm0, xmm12, xmm2
59 vfmsub231sd xmm0, xmm13, xmm3
60 vmulsd xmm0, xmm8, xmm0
61 vfmadd231sd xmm0, xmm6, xmm14
62 vfmadd231sd xmm0, xmm4, xmm11
63 vmulsd xmm11, xmm8, xmm10
64 vfmadd231sd xmm11, xmm6, xmm12
65 vfmadd231sd xmm11, xmm4, xmm13
66 vmulsd xmm8, xmm8, xmm1
67 vfmadd231sd xmm8, xmm3, xmm6
68 vfmadd231sd xmm8, xmm2, xmm4
69 vmulsd xmm1, xmm10, xmm1
70 vfmadd231sd xmm1, xmm12, xmm3
71 vfmadd231sd xmm1, xmm13, xmm2
72 vfmadd213sd xmm1, xmm5, xmm15
73 vfmadd231sd xmm1, xmm9, xmm8
74 vfmadd231sd xmm1, xmm7, xmm11
75 call r12
```

Fused Multiply-Add (FMA) 3/

Cargo.toml

Add an `fma` feature

```
[features]
default = []          # Not on by default!
fma = []
python = ["fma"] # Enable for python
```

Code

Use FMA if enabled

```
// Horner's method
#[cfg(not(feature = "fma"))]
{
    y0 + t * (c1 + t * (c2 + t * c3))
}
#[cfg(feature = "fma")]
{
    c3.mul_add(t, c2).mul_add(t, c1).mul_add(t, y0)
}
```

Perf

Profit

```
grid/Linear Regular 100x1D MAXDIMS=1,
time:    [320.70 ns 321.86 ns 323.31 ns]
thrpt:   [309.30 Melem/s 310.69 Melem/s]
:
time:    [-13.441% -12.630% -11.896%]
thrpt:   [+13.502% +14.456% +15.528%]
Performance has improved.
```

Rust after Code

The myth of premature optimization.

Telling rustc what you know about the user

Profile-Guided Optimization (PGO) for Python Extensions 1/

Python extensions are good for adoption & distribution

- 10k downloads/month vs. 20k all-time, in this case

General guidelines for Python Extensions

- Use pyo3+maturin with ABI3; target 2013+ (x86-64-v3)
- Publish as both a Rust crate *and* a Python library
- `maturin generate-ci`
- Document units-of-measure & cite sources!!
- Do PGO if you can phrase a workload

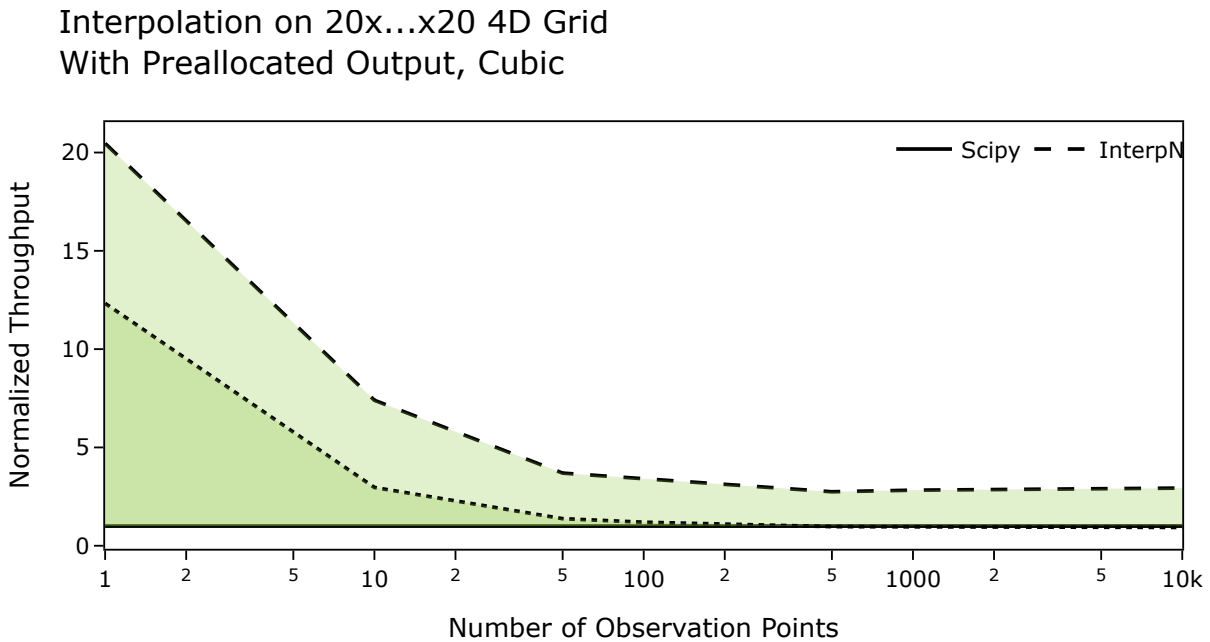
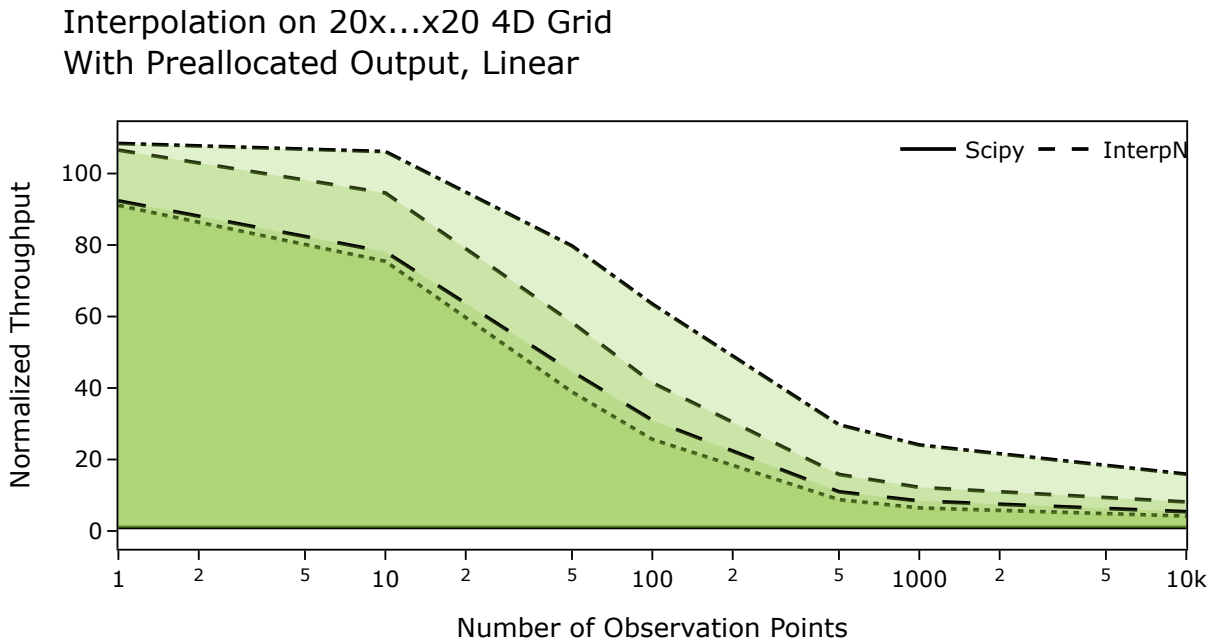
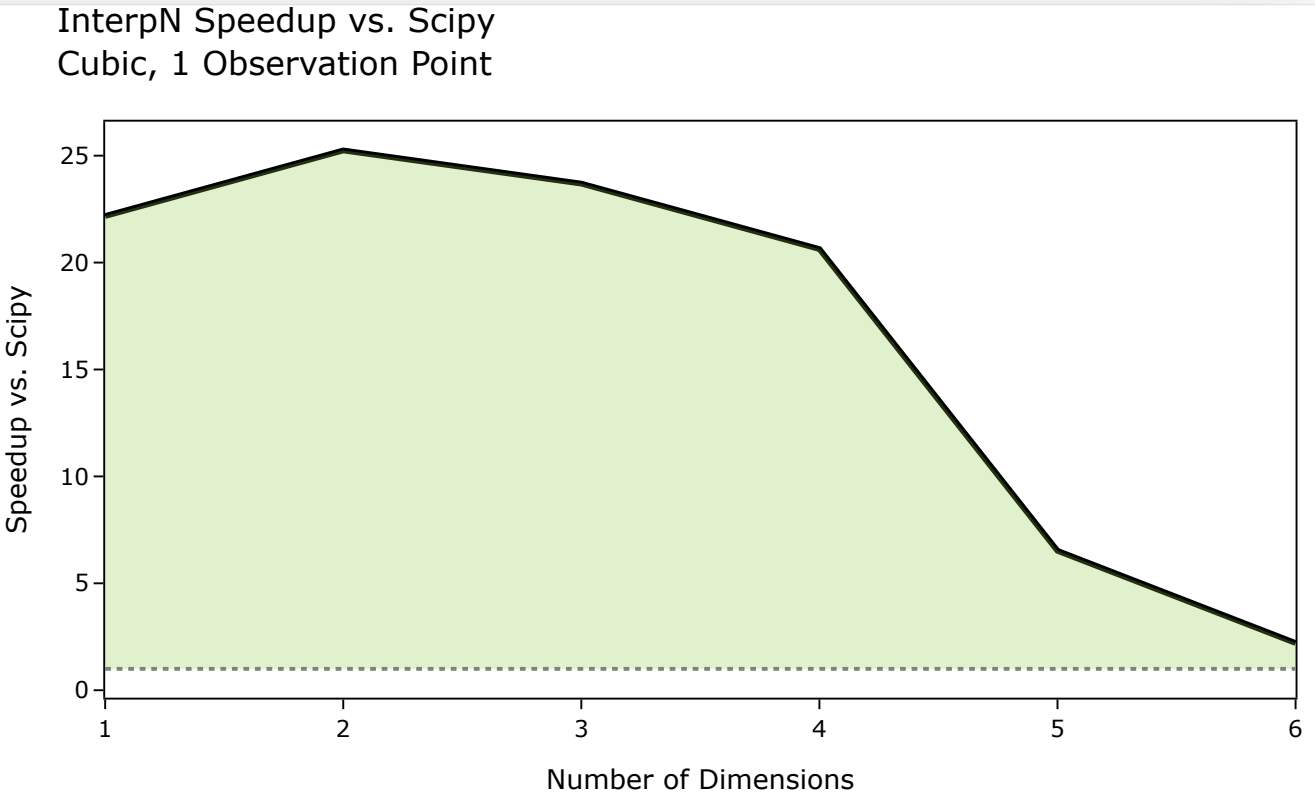
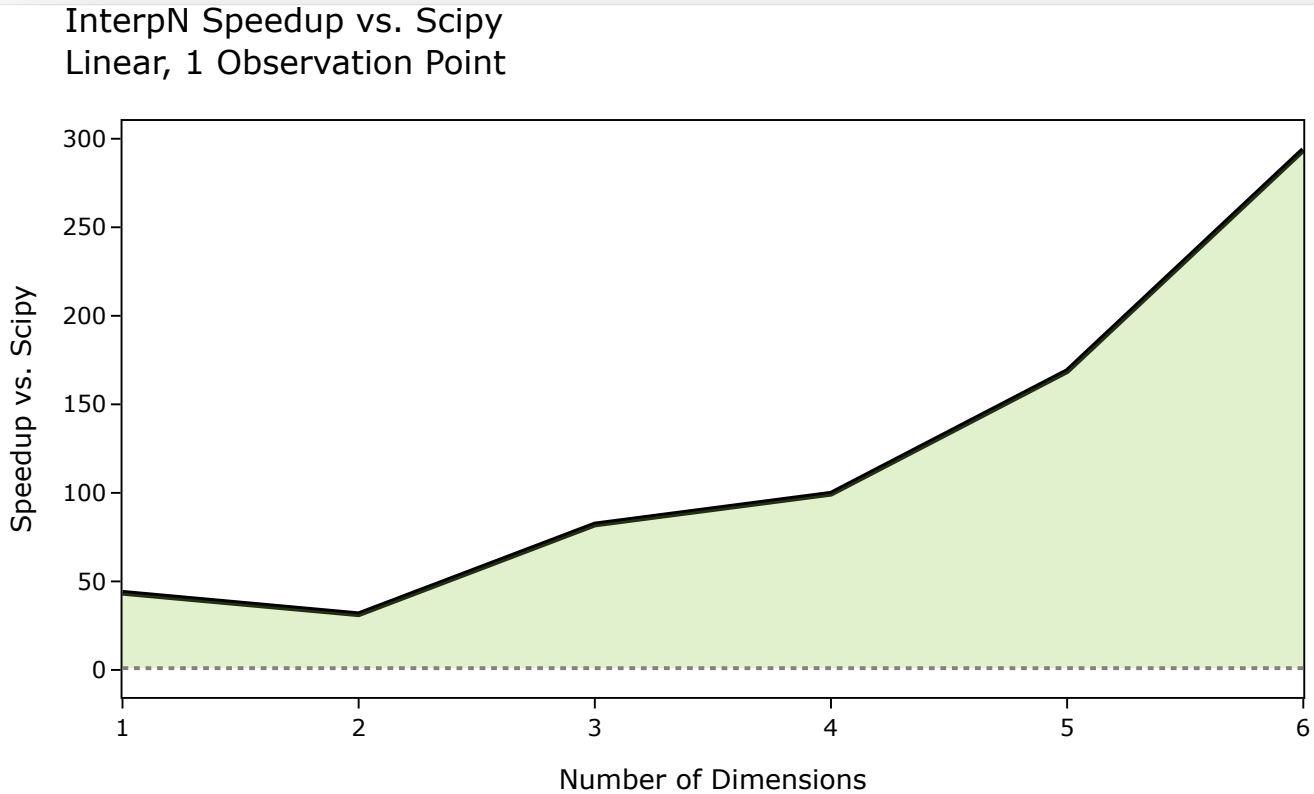
PGO Procedure

1. Build instrumented (just extra rustc args via maturin!)
2. Run PGO workload (just a python script!)
3. Process profile outputs (just call llvm on some files!)
4. Build optimized (just extra rustc args via maturin!)

Bottom line: 15-30% speedup for most-used cases.

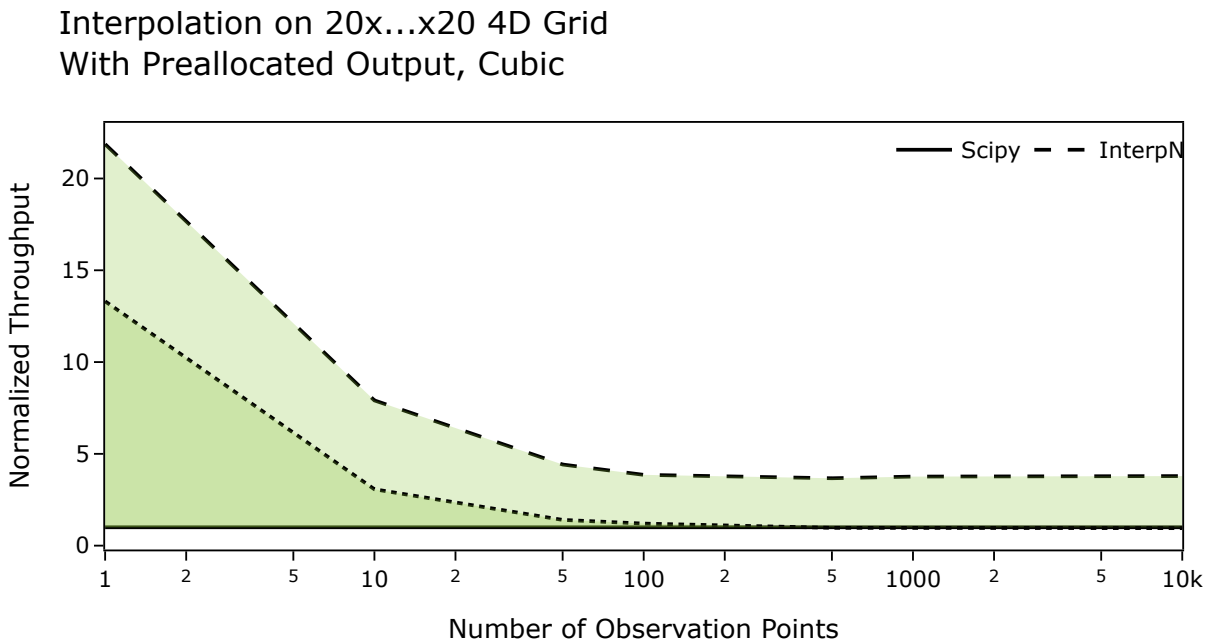
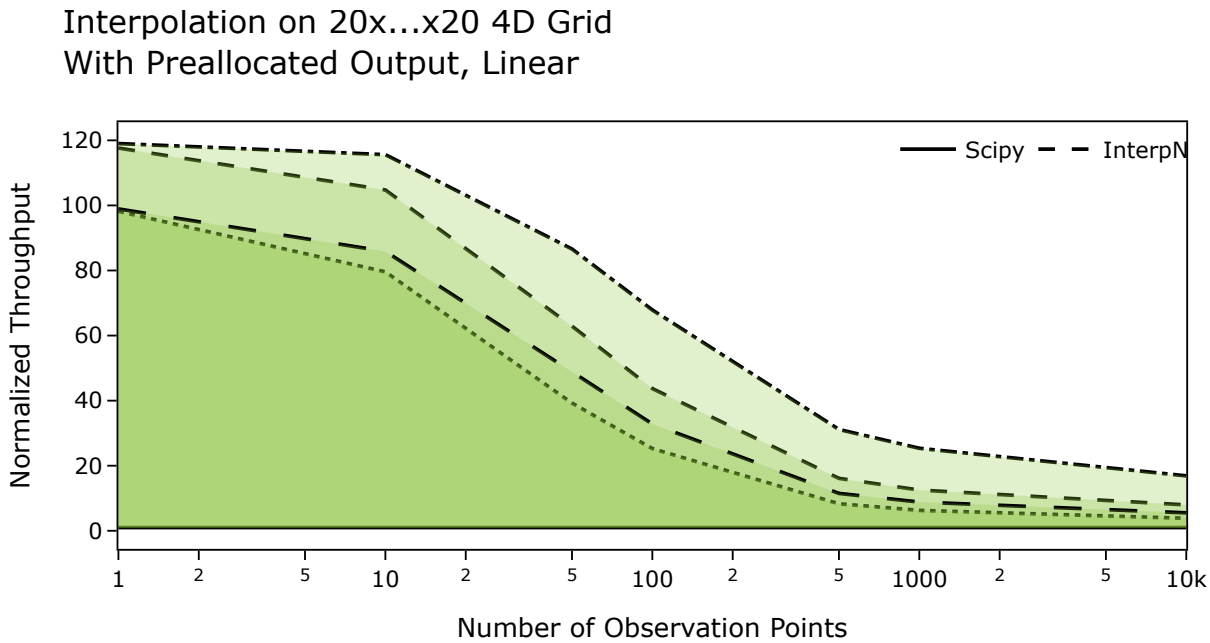
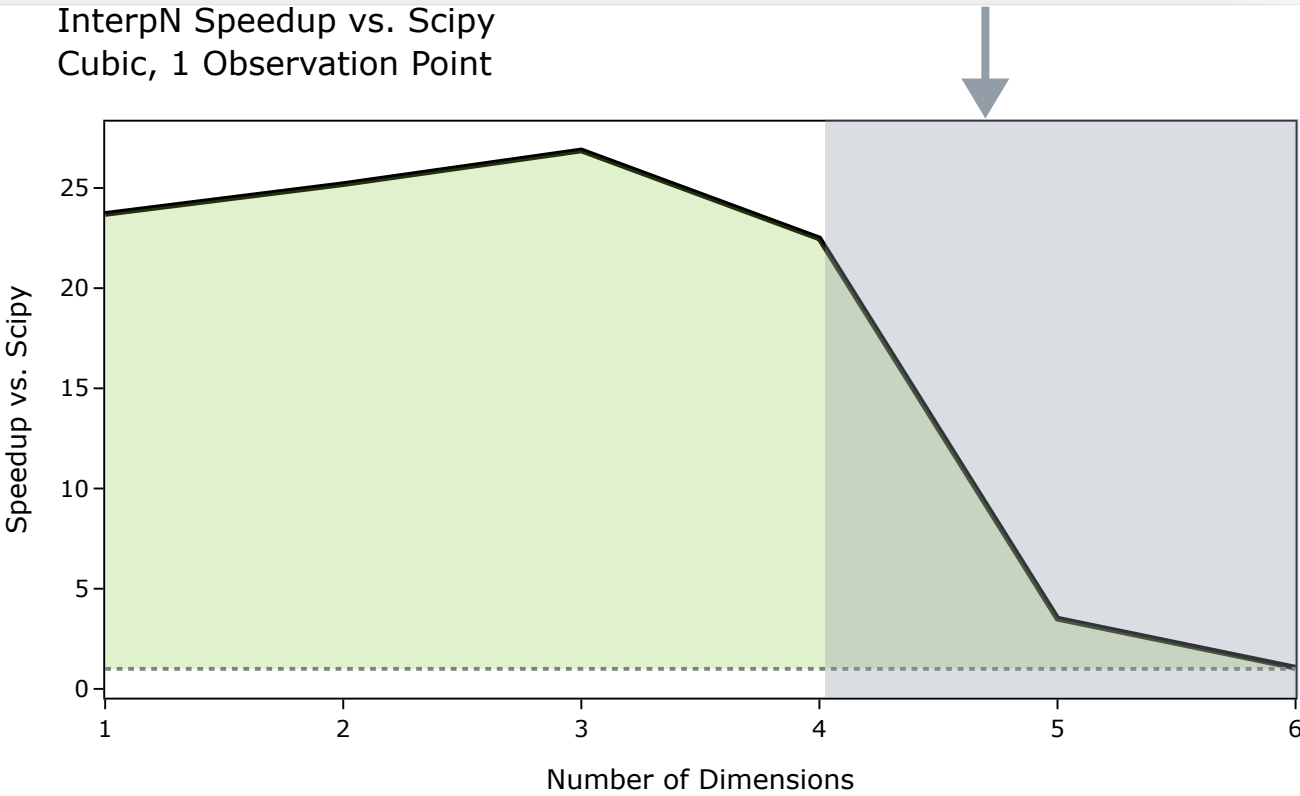
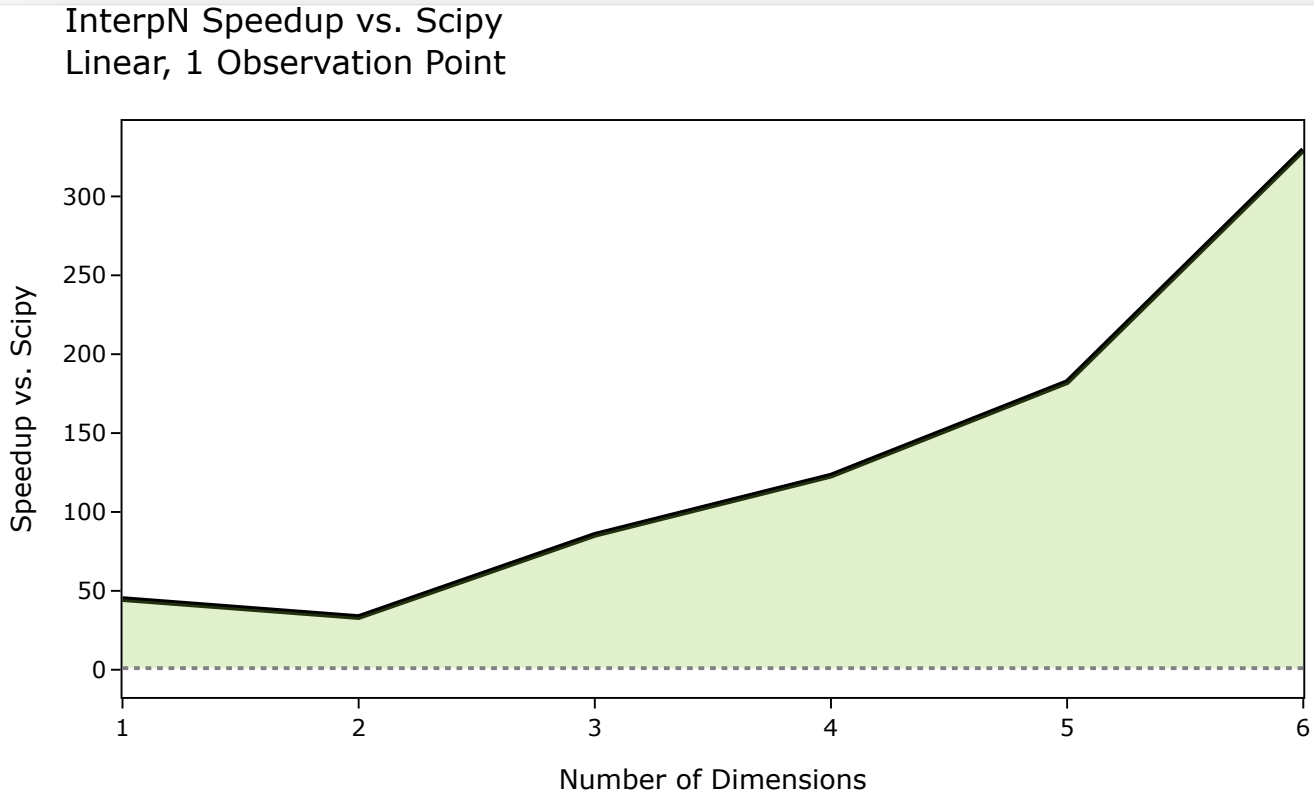
Before

Profile-Guided Optimization (PGO) for Python Extensions 2/



After

Profile-Guided Optimization (PGO) for Python Extensions 3/

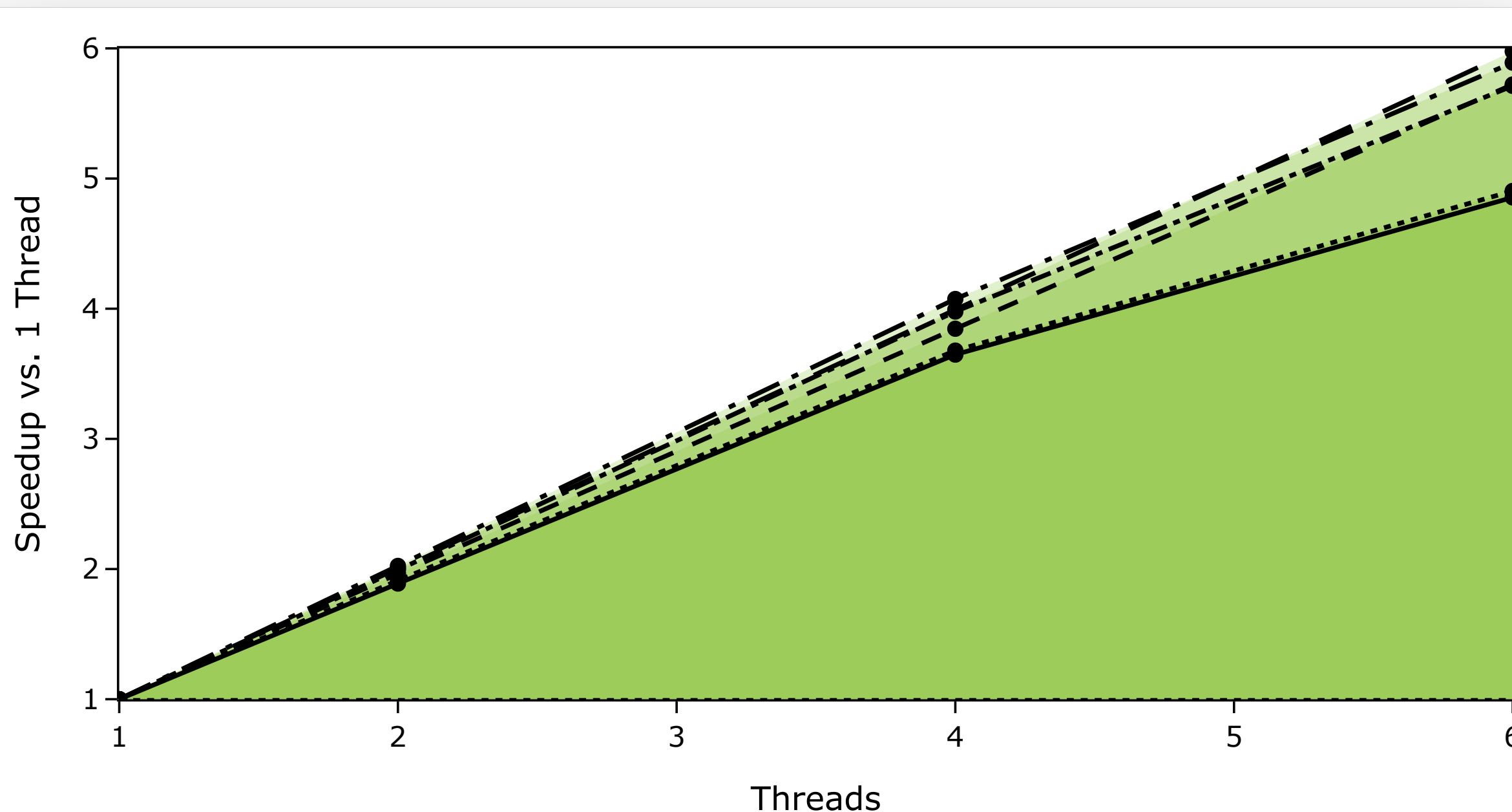


Do this *last*, not *first*!

Thread Parallelism 1/

Near-linear speedup; limited only by thread spawning overhead & CPU heating

- Chunk slices for each thread → no allocation, message-passing, or locks
- Use global lazy static to check number of physical cores once and never again
- Only thread if it's faster than single-core
- Don't voluntarily yield the thread



Benchmark case
representative of a
10-megapixel image.

Recipes, or it didn't happen

Thread Parallelism 2/

- ✓ No allocation
 - Slice reference inputs
 - Stack-only Result output
- ✓ No contentious locks
 - `PHYSICAL\_CORES` may spin briefly on first access
 - Atomic load only after init
- ✓ Tiny amount of overhead for chunking only

Don't just parallelize over scalar calcs!

Vectorize over chunks.

```
use rayon::prelude::*;
use std::sync::LazyLock;

/// Populated once on first access, then never again.
/// No lock used for access after initialization!
static PHYSICAL_CORES: LazyLock<usize> = LazyLock::new(num_cpus::get_physical);

/// Vector-parallel variant of [solid_angle_tetrahedron_scalar]
#[inline] // Enable cross-crate inlining
pub fn solid_angle_tetrahedra_par(
    tetrahedra: &[[[f64; 3]; 4]],
    out: &mut [f64],
) -> Result<(), &'static str> {
    // Chunk inputs. Only use real cores!
    let num_cores = rayon::current_num_threads().min(*PHYSICAL_CORES);
    let num_chunks = 1024.min(out.len() / num_cores);
    let (tet_chunks, out_chunks) = (
        tetrahedra.par_chunks(num_chunks),
        out.par_chunks_mut(num_chunks),
    );

    // Do vector calculations over each chunk in parallel
    (tet_chunks, out_chunks)
        .into_par_iter()
        .try_for_each(|(tetc, outc)| solid_angle_tetrahedron(tetc, outc))?;

    Ok(())
}
```

vector-parallel formula

vector formula

Thread Parallelism 3/

- lscpu (native cpu info)
- pidstat (track thread preemption live)
- cachegrind (simulate cache misses)
- strace (track syscalls live)
- perf stat (estimate cache misses live)
- cargo-call-stack (inspect stack usage)
- stats_alloc (instrument global allocator)

```
~$ lscpu
...
Caches (sum of all):
L1d:      192 KiB (6 instances)
L1i:      192 KiB (6 instances)
L2:       3 MiB (6 instances)
L3:      32 MiB (2 instances)
```

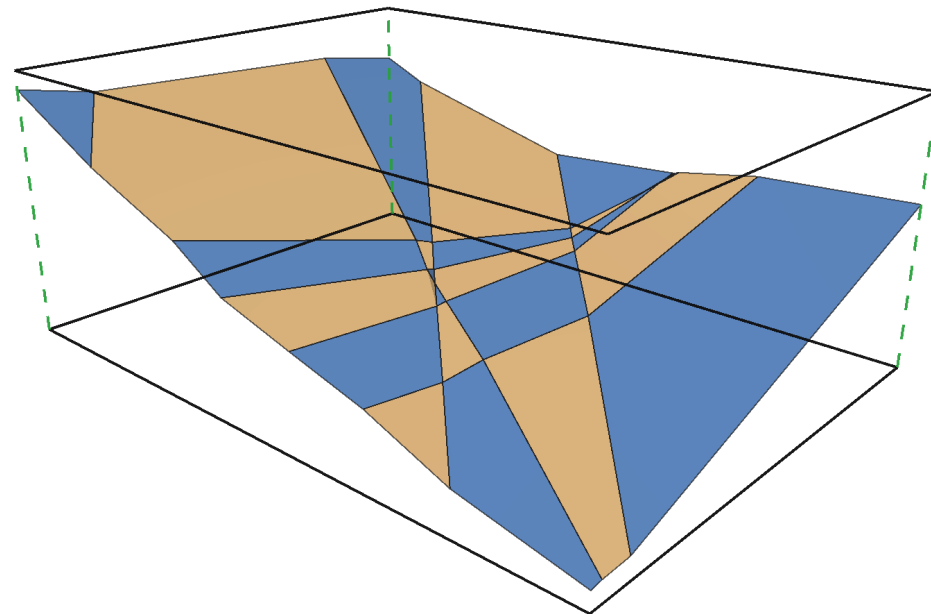
```
~$ pidstat -w -p -t <pid>
12:01:58 AM UID  TGID TID cswch/s nvcschw/s Command
12:01:58 AM 1000 2983046 - 0.01 0.00 multi_daq
12:01:58 AM 1000 - 2983046 0.01 0.00 |__multi_daq
12:01:58 AM 1000 - 2984161 0.21 0.00 |__tsdb-dispatcher
12:01:58 AM 1000 - 2984162 0.70 0.01 |__csv-dispatcher
```

```
# Add these to configure a specific cache size:
# <size>,<associativity>,<line size>
# --l1=32768,8,64\
# --D1=32768,8,64\
# --LL=8388608,16,64\
~$ valgrind --tool=cachegrind --cachegrind-out-file=./cachegrind.out <COMMAND>
~$ kcachegrind ./cachegrind.out # visualize
~$ cg_annotate --auto=yes ./cachegrind.out # show line numbers
~$ cg_annotate --source=yes ./cachegrind.out # annotate source code
```

```
~$ sudo perf stat -e cache-misses,cache-references,branches,branch-misses su - <USER> -c "<COMMAND>"
Performance counter stats for 'su - <USER> -c <COMMAND>':
 262,051,652  cache-misses          # 6.88% of all cache refs
3,807,478,401  cache-references
93,183,164,713  branches
923,190,394  branch-misses          # 0.99% of all branches
```

```
~$ strace -tt -f --syscall-times=ns -C <COMMAND>
# Track syscalls
# with forked threads (-f)
# with microsecond timestamps (-tt)
# with nanosecond-resolution durations (--syscall-times=ns)
# with a summary table at the end (-C)
```

Future



ReLU networks are just unstructured piecewise-linear interpolation!

Try Pitch

GPU/TPU Offloading

Cross-platform GPU acceleration via Vulkan SPIRV is possible.

- Example all-rust GPU compute kernel repo: <https://github.com/jlogan03/krnل-example>
- Updates to `krnl` for latest `rust-gpu`: <https://github.com/jlogan03/krnل>
- Exploring gaps in SPIRV lowering capabilities
 - Already almost works out-of-the-box, since InterpN is no_std!
 - Result<T,E>, iterators, and some operations not supported.
 - Some modifications to InterpN are required (return a flag result, use more range loops).

Point-Cloud Methods

- Unstructured nearest-neighbor / radial-basis
 - Naive implementation is easy; good implementation will take careful thought.
 - Shares machinery with Barnes-Hut solvers, ND Cheby node interpolation, NURBS, finite-element postprocessing, etc.
- Neural network
 - NNs are just flattened interpolators! Classification is an after-thought.
 - Have to separate training (with std) from evaluation (no_std) and store weights as static.
 - Training methods must enforce constraints (monotonicity, etc.) to be useful.

Links

- InterpN @ v0.11.0 | [github](#) | [crates.io](#) | [pypi](#) | [PGO workflow](#)
- This slide deck | [pitch.io](#) | [github](#) | [godbolt w/ FMA](#) | [godbolt w/o FMA](#)
- The blog post | [jlogan.dev](#)

Further Reading

- [Scalability! But at what COST?](#)
- [LLVM's autovectorizer docs](#)
- Jakub Beránek's [blog post](#) about cargo-pgo
- Nick Wilcox's [auto-vectorizer tutorial](#)
- Predrag Gruevski's [blog post](#) about optimizing cargo-semver-checks
- David Hewitt's [talks](#) about PyO3 (any of them, really)
- Miguel Raz's [talk](#) about SIMD for Sav-Gol filters
- Carl Kadie's [blog post](#) about guidelines for scientific libraries in Rust
- Ryan Levick's talk [Rust before Main](#)
- [Herbie](#) floating-point expression compiler
- Jane Street [blog post](#) about neural network interpolators

Questions?

Acknowledgements

- Predrag Gruevski - mentor
- Clément Robert - contributor
- Yann Simon - reference CPUs
- Herold Aptroot - FMA latency
- Will Glyn - Herbie

Hardware Information

Model
Micro-Star International
Co., Ltd. MS-7C56

Memory
64.0 GiB

Processor
AMD Ryzen™ 5 3600 × 12

Graphics
NVIDIA GeForce RTX™ 3070

Disk Capacity
500.1 GB

Software Information

Firmware Version
A.30

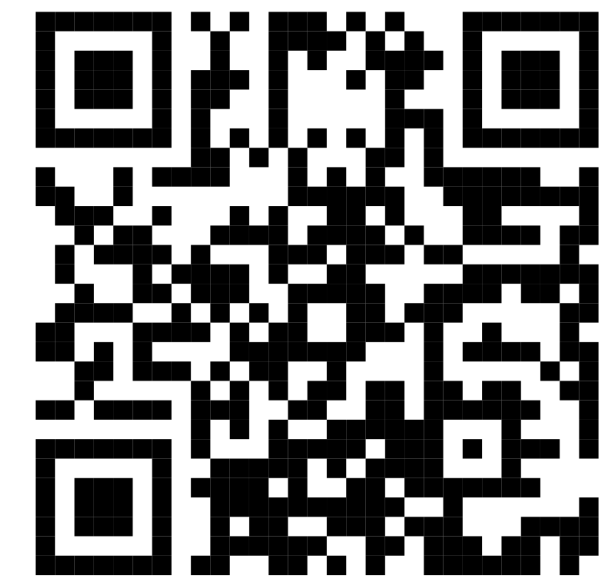
OS Name
Ubuntu 24.04.3 LTS

OS Type
64-bit

GNOME Version
46

Windowing System
X11

Kernel Version
Linux 6.8.0-90-generic





Want to make a presentation like this one?

Start with a fully customizable template, create a beautiful deck in minutes, then easily share it with anyone.

Create a presentation (It's free)